

Veritrust: Quality-Gated Synthetic Data Generation for Regulated Industries

One filtering, masking, and evaluation methodology across two generation substrates and two record types — instruction fine-tuning and construction-true de-identification

Patrick Hill, M.S.
Founder & Principal AI/ML Systems Architect
PBH Applied Systems LLC

July 2026

Abstract

Synthetic training data is now the operating standard at the frontier of model development, but the public synthetic-data ecosystem remains opaque: datasets ship with little documentation of how they were generated, what was filtered out, or what compliance posture they carry, and de-identification corpora — the labeled data used to train the systems that redact clinical and legal records — frequently carry labels that cannot be verified. This paper documents Veritrust, a single quality-gated methodology — semantic hallucination filtering, regulation-specific compliance masking, construction-true de-identification labeling, automated evaluation, and provenance-stamped delivery — deployed across two complementary generation substrates and producing two orthogonal record types. Veritrust Studio runs generation locally on a single consumer GPU; Veritrust Cloud offloads generation to a Modal A10G served by vLLM; both import an identical shared core and differ only in where tokens are produced. The same methodology emits instruction fine-tuning records and construction-true de-identification records, whose span labels are correct by construction because the personal-data values are injected at positions the pipeline itself records, allowing the dataset to serve as ground truth rather than as another fallibly-tagged corpus. The central engineering contribution is the exposure and recovery of a generation defect: a general-purpose model emits fill-in-the-blank skeletons rather than completed records, and a narrow template-leak gate had been counting those skeletons as accepted output at a false 95% yield. Widening the gate exposed a true 35% baseline, which four successive fixes — a document-type change, a recalibrated hallucination threshold, value-packet-first generation, and a batched repair pass — recovered to a validated 86.9%. The methodology was validated on two log- and manifest-verified ten-thousand-record medical de-identification runs, one per substrate, with comparable yield (86.9% Studio, 85.5% Cloud) and a hybrid regex-plus-learned-detector micro-averaged F1 of 0.964 against 49,344 construction-true ground-truth spans. The economics were measured rather than projected: the Studio run delivered ten thousand records in roughly 17.8 hours for about \$0.58 in electricity, and the Cloud run in roughly 3.4 hours for \$3.60 in metered GPU credits. Results are reported as measured, bounds are stated plainly, and the value of the methodology is the downstream cost and risk it removes for a regulated buyer who receives an attested, construction-true, span-labeled deliverable rather than raw generated tokens.

Contents

1 Executive Summary

3

2	The Synthetic-Data Quality Problem	4
2.1	The Legal Exposure of Web-Scraped Training Data	5
2.2	The Regulatory Wall	6
2.3	The Financial Exposure	8
2.4	One Methodology, Two Substrates, Two Record Types	9
2.5	The Layer That Is Missing	9
3	Architecture Overview	10
3.1	The Shared Core and the Two Substrates	10
3.2	The Core Subpackages	12
3.3	The Two Substrate Wrappers	13
3.4	The Two Record Types	13
3.5	Runtime Flow, Evaluation, and Deliverables	13
4	The Hallucination Gate	15
4.1	Hallucination as Instruction-Output Drift	15
4.2	The Dual-Signal Decision Rule	15
4.3	Recalibrating the Threshold: The 0.45 Unified Floor	16
4.4	Production Validation	17
4.5	Substrate Independence of the Gate	17
5	The Compliance and De-identification Architecture	17
5.1	Compliance as a Gate, Not a Disclaimer	18
5.2	Layered PII Masking	18
5.3	Construction-True De-identification Labeling	19
5.4	Value-Packet Generation and Token Injection	19
5.5	The Learned Detector Tier	20
5.6	The Record-Level Acceptance Model	20
6	The Template-Leak Gate and Yield Recovery	21
6.1	The Skeleton Defect	21
6.2	The Template-Leak Gate	21
6.3	The Four-Fix Recovery	22
6.4	Why This Is the Central Result	23
7	The Evaluation Pipeline and the Attested Report	23
7.1	Evaluation as a Deliverable	23
7.2	Reconciliation	24
7.3	De-identification Detector Evaluation	24
7.4	The Detector Evaluation Is Configurable and Substrate-Independent	26
8	Results	26
8.1	The Two Production Runs	26
8.2	Substrate Behavior and Rejection Composition	27
8.3	De-identification Label and Detector Quality	28
8.4	The Instruction Fine-Tuning Record Type at Production Scale	28
8.5	The De-identification Detector Across the Full Type Vocabulary	28
8.6	What the Results Establish	29
9	Throughput and Economics	30
9.1	The Measured Ten-Thousand-Record Comparison	30
9.2	Owned Infrastructure Versus Metered Rental	30
9.3	The Substrate Decision	31
9.4	The Value of Attestation	32

10	Limitations	32
10.1	Injected Value Fidelity	32
10.2	The Macro-Averaged Detector Score	33
10.3	Pattern-Based Detection and Masking Cannot Be Proven Complete	33
10.4	Threshold Calibration Is Empirical, Not Adaptive	33
10.5	Substrate and Scale Bounds	34
11	Appendix	34
11.1	Configuration Reference	34
11.2	Run Inventory	35
11.3	Environment Specification — Local Substrate (Veritrukt Studio)	36
11.4	Environment Specification — Cloud Substrate (Veritrukt Cloud)	36
11.5	Glossary	37
12	References	37

1 Executive Summary

Synthetic training data has become the operating standard at the frontier of model development, but the public synthetic-data ecosystem has not matured with it. Datasets are published with little documentation of how they were generated, what was filtered out, or what compliance posture they carry — and in the case of de-identification data, the labeled corpora used to train and benchmark the systems that redact clinical and legal records, the labels themselves are frequently unverifiable. This gap has become untenable at exactly the moment three forces have converged: copyright litigation that has made the provenance of training data a liability rather than a settled question; privacy regulation — GDPR, HIPAA, CCPA — under which scraped personal data carries unbounded and architecturally irreversible exposure; and the per-token economics of frontier generation APIs, which make the iterative development of a quality-attested dataset expensive enough to discourage it. Together these define a clear need: synthetic data produced under a controlled, documented, compliance-audited methodology, with quality and provenance that survive an audit.

This paper documents Veritrukt, a methodology that meets that need, structured along two orthogonal dimensions. The first is the substrate. A single quality-gated pipeline — semantic hallucination filtering, regulation-specific compliance masking, construction-true de-identification labeling, automated evaluation, and provenance-stamped delivery — is deployed across two complementary generation substrates that share an identical core and differ only in where generation tokens are produced. Veritrukt Studio runs generation locally on a single consumer GPU, where near-zero marginal cost makes it the substrate for development and for low-volume or flexible-timeline production. Veritrukt Cloud runs the identical methodology with generation offloaded to a cloud GPU service, where higher throughput makes it the substrate for high-volume delivery under deadline. The second dimension is the record type. The same methodology produces both instruction fine-tuning records, in which an instruction is paired with a quality-gated response, and construction-true de-identification records, in which text is paired with span-level labels marking the location and type of every personal-data element it contains. Frontier generation APIs are treated throughout as a neutral cost-and-capability baseline, not an adversary: they generate raw tokens, while the value this paper documents is the production layer that turns generated tokens into an attested, compliant deliverable.

The central technical contribution is the construction-true de-identification record type and the engineering that made it viable. Because its span labels are the product rather than an annotation applied after the fact, they are correct by construction — the personal-data values are injected into the text at positions the pipeline itself records — which lets a Veritrukt de-

identification dataset serve as ground truth rather than as another fallibly-tagged corpus. Making this work at generation time required exposing and resolving a defect that a naive quality gate had been silently accepting: a general-purpose model, asked for a completed record, emits fill-in-the-blank skeletons rather than finished documents. A narrow template-leak gate had been counting those skeletons as accepted output, reporting a false 95% yield; widening the gate to recognize every skeleton style collapsed the apparent yield to a true 35%, and four successive fixes — a document-type change, a recalibrated hallucination threshold, value-packet-first generation, and a batched repair pass — recovered it to a validated 86.9%. This episode is the clearest evidence for the paper’s recurring claim that the measurement infrastructure is itself the product.

The methodology was validated on two log- and manifest-verified ten-thousand-record medical de-identification runs, one on each substrate. Both delivered comparable yield — 86.9% on Studio, 85.5% on Cloud — with identical rejection ordering, confirming that the substrate is an operational choice rather than a quality determinant. A hybrid regex-plus-learned-detector evaluation, scored against 49,344 construction-true ground-truth spans, achieved a micro-averaged F1 of 0.964, clearing 0.95 on every identifier type carrying meaningful frequency in a medical corpus. The economics were measured, not projected: the Studio run delivered ten thousand records in roughly 17.8 hours for about \$0.58 in electricity, and the Cloud run delivered the same volume in roughly 3.4 hours for \$3.60 in metered GPU credits — the cloud substrate roughly five times faster at a still-trivial absolute cost, and both an order of magnitude or more below the raw-token cost of a frontier API.

The methodology’s value, ultimately, is not the savings on token generation but the cost and risk it removes downstream. A regulated buyer receives a construction-true, span-labeled dataset accompanied by a machine-readable report covering rejection reasons, a reconciliation of the run’s own counts, and a detector evaluation scored against known-correct labels — the verification work the buyer would otherwise have to perform or absorb as risk. The paper states its limitations as plainly as its results: the injected value fidelity carries a measured shortfall, the detector’s macro-averaged score reflects untested low-support types, pattern-based detection cannot be proven complete, the thresholds are calibrated rather than self-adjusting, and the quantitative results characterize one domain and one record type at the ten-thousand-record scale. What the evidence establishes is the core claim: one quality-gated methodology, deployed across two complementary substrates and producing two record types, yields attested, compliant, construction-true datasets at a cost and throughput that make defensible synthetic data practical for regulated industries.

2 The Synthetic-Data Quality Problem

Synthetic training data has moved from research curiosity to operating standard. The transition is most visible at the frontier: OpenAI, Google, Meta, and Anthropic all use synthetic or model-generated data in training, fine-tuning, post-training, alignment, or safety-evaluation workflows for major model systems (Anthropic, 2026; Gemini Team, Google, 2025; Guan et al., 2024; Meta AI, 2024; Wiggers, 2024). NVIDIA — the hardware vendor whose GPUs train every major model — has gone further and released Nemotron-4 340B, a model family explicitly designed for synthetic data generation, alongside a disclosure that more than 98% of the data used in its own model alignment process is synthetically generated (Adler et al., 2024). The category is no longer experimental. It is the default modern training methodology.

Yet the public synthetic-data ecosystem does not reflect this maturity. Datasets are published with minimal documentation of how they were generated, what was filtered out, what compliance posture they assume, or what quality guarantees they carry. A buyer evaluating a synthetic dataset for use in a regulated commercial setting — clinical NLP, legal-tech, financial services — typically finds a dataset card with row counts, a license, and little else. The

generation pipeline is opaque. The rejection criteria are unstated. The compliance coverage is unaudited. And in the specific case of a de-identification dataset — text annotated with the locations of personal data, used to train and benchmark the models that redact clinical and legal records — the labels themselves are frequently unverifiable, because a dataset labeled by the same class of tool it is meant to evaluate cannot establish ground truth.

This gap exists because three forces have converged simultaneously, and the open ecosystem has not yet caught up to any of them: a wave of copyright litigation against the use of scraped web data for model training; a tightening regulatory environment in which scraped personal data carries unbounded legal exposure; and a cost structure for synthetic data generation in which the per-token rates of frontier generation APIs make the iterative development of a quality-attested dataset expensive enough to discourage it. Each of these is independently sufficient to motivate the adoption of compliance-documented, quality-gated synthetic data produced under a controlled methodology. Taken together, they define the market that the present work addresses.

The work documented in this paper addresses that market through a single quality-gated methodology — hallucination filtering, regulation-specific compliance masking, automated evaluation, and independent audit — deployed across two complementary generation substrates and applied to two distinct record types. Veritrukt Studio runs the methodology on a local consumer-GPU workstation, where near-zero marginal cost makes it the natural substrate for development and for low-volume or flexible-timeline production. Veritrukt Cloud runs the identical methodology on cloud GPU infrastructure, where higher throughput makes it the natural substrate for high-volume production under delivery deadlines that local generation cannot meet. The two substrates are not competing alternatives; they are complementary deployments of one methodology, and the choice between them is an operational decision driven by the volume and timeline of a given engagement. Across both substrates, the same methodology produces two orthogonal record types: instruction fine-tuning records, in which an instruction is paired with a quality-gated model response, and construction-true de-identification records, in which text is paired with span-level labels marking the location and type of every personal-data element it contains. The two record types share the generation, quality, masking, and evaluation infrastructure and differ only in what they emit and how they are validated. Frontier generation APIs are referenced throughout this paper as a cost and capability baseline, not as an adversary: they are a legitimate way to generate tokens, but they generate raw, ungated, uncompliant output, and the layer this paper documents — the filtering, masking, evaluation, and audit that turn generated tokens into an attested deliverable — is absent from a raw API call regardless of which model serves it.

2.1 The Legal Exposure of Web-Scraped Training Data

For most of the last decade, the assumption underlying commercial model training was that web-scraped data was fair game under U.S. fair-use doctrine. That assumption is no longer safe to make. The legal landscape has shifted materially over the past eighteen months, and any organization training models on uncurated scraped data now operates under a substantial and growing liability footprint.

The most consequential development is *Bartz et al. v. Anthropic*. In June 2025, the U.S. District Court for the Northern District of California issued a split summary judgment: training on legally acquired books was held to be fair use, but downloading and storing copies from shadow libraries was not (Authors Guild, 2025). The piracy ruling led directly to a \$1.5 billion settlement — the largest copyright settlement in U.S. history (Wolters Kluwer, 2025). The final fairness hearing was held on May 14, 2026, with claims filed against approximately 440,490 of the 482,460 works on the certified works list, a 91.3% claims rate (Authors Alliance, 2026). The settlement establishes a per-title compensation framework of roughly \$3,000 per work and sets the operating precedent that AI developers can be held liable for

the *provenance* of their training data, not merely for whether the training itself qualifies as transformative use.

The implication for downstream users is direct. Plaintiff firms have begun developing what one commentator has termed the “Shadow Library Strategy”: skip the unresolved question of whether AI training is fair use, and litigate the piracy angle instead (Wolters Kluwer, 2025). Most new book-related copyright lawsuits against AI companies — including ongoing claims against Meta in *Kadrey v. Meta Platforms* — now allege downloads from shadow libraries (Norton Rose Fulbright, 2026). Any organization that fine-tunes a foundation model on a corpus of indeterminate provenance inherits this exposure.

In parallel, discovery practice in *In re: OpenAI, Inc. Copyright Infringement Litigation* has established that AI companies cannot keep their training logs and outputs private. On January 5, 2026, U.S. District Judge Sidney Stein affirmed an order compelling OpenAI to produce its entire 20-million-log sample, not a curated subset (National Law Review, 2026). On March 9, 2026, the court further compelled production of an additional 78 million and 10 million logs (Norton Rose Fulbright, 2026). The “Stein Standard” — that broad discovery prevails over developer-controlled search restrictions when copyright infringement is alleged — is now operative precedent. Every prompt-response pair preserved in an organization’s training infrastructure is potentially discoverable evidence.

The picture is not entirely one-directional. The UK High Court’s November 2025 decision in *Getty Images (US) Inc. v. Stability AI Limited* held that AI model weights themselves are not “infringing copies” under UK law, because the weights do not store or reproduce the underlying training images (Latham & Watkins, 2025; Mayer Brown, 2025). The ruling provided a narrow win for AI developers on the secondary-infringement theory. However, the *Getty* decision turned heavily on the factual concession that training had occurred outside the United Kingdom, and Getty had withdrawn its primary infringement claims before trial (Ropes & Gray, 2026). The case did not resolve whether training itself constitutes infringement — only that downstream distribution of a trained model is not, on these facts, a secondary infringement event. European jurisprudence has moved in the opposite direction: in November 2025, the Munich Regional Court held in *GEMA v. OpenAI* that training GPT-4 on copyrighted German song lyrics violated German copyright law, among the first major European rulings against an AI developer on training-data grounds (AI Lawsuit Tracker, 2026).

The practical reality for any organization fine-tuning models for commercial deployment is that the legal status of web-scraped training data is contested, jurisdictionally fragmented, and trending toward greater rather than lesser exposure. Indemnification clauses in vendor agreements increasingly reflect this — buyers now ask whether training data provenance can be documented, and a vendor who answers “we scraped the web” is at an active commercial disadvantage. Synthetic data generated under controlled provenance, with the generating model identified and the generation process documented, eliminates this exposure entirely. There is no shadow library to be discovered, no third-party rightsholder to assert a claim, no unresolved fair-use question to litigate. The data was generated, by a known model, under conditions the publisher can attest to.

2.2 The Regulatory Wall

Copyright is one source of training-data risk. Privacy regulation is the other, and for organizations operating in regulated industries it is the more immediately binding constraint.

Under the EU General Data Protection Regulation (GDPR), Article 5 imposes the principles of data minimization, purpose limitation, and storage limitation; Article 6 requires a valid lawful basis — consent, contract, legal obligation, vital interests, public task, or legitimate interest — for any processing of personal data (European Parliament & Council of the European Union, 2016). Web-scraped data for AI training struggles against every one of these requirements si-

multaneously. Consent is practically impossible to obtain from the unidentified rightsholders in a scraped corpus (Berberich, 2025). Purpose limitation is breached when data collected for one purpose is repurposed for model training without a new lawful basis. The European Data Protection Board’s December 2024 Opinion 28/2024 and the French CNIL’s June 2025 guidance both acknowledge that legitimate interest under Article 6(1)(f) can in principle support AI training, but only after a three-step assessment that most web-scraping operations cannot satisfy (Skadden, 2025; Orrick, 2025).

The architectural problem with web-scraped training data under GDPR is more fundamental than the lawful-basis question. Once personal data has been incorporated into model weights, it cannot be selectively removed. Article 17 — the right to erasure — assumes that personal data exists in a structured store from which it can be deleted. A neural network is not such a store. A June 2025 University of Tübingen study established that large language models memorize between 0.1% and 10% of training data verbatim (PPC Land, 2026), which means scraped personal data persists in deployed models in a form that may be retrievable but cannot be erased without retraining. This is not a compliance gap that can be closed with policy changes. It is a structural incompatibility between the architecture of foundation models and the rights regime under which European personal data is governed.

Enforcement has been escalating. Italy’s data protection authority, the Garante, issued a €15 million fine against OpenAI in December 2024 and banned DeepSeek from operating in Italy in January 2025 (PPC Land, 2026). It is worth noting that an Italian court in Rome annulled the penalty entirely in March 2026, ruling in favor of OpenAI (Wilson Sonsini Goodrich & Rosati, 2026). The Irish Data Protection Commission obtained X’s voluntary suspension of EU user data processing for AI training in August 2024 (Norton Rose Fulbright, 2025). Clearview AI has received fines ranging from €250,000 to €30.5 million across France, Greece, Italy, the Netherlands, and Sweden, with several authorities ordering deletion of scraped datasets (IAPP, 2025). The European Commission’s November 2025 Digital Omnibus initiative proposes amendments that would explicitly establish AI training as a legitimate interest under a new Article 88c, but Estonia, France, Austria, and Slovenia have formally opposed the changes, and the outcome remains politically contested (PPC Land, 2026).

The U.S. healthcare context is more constrained still, and it is the context in which the de-identification record type this paper documents is most directly motivated. The Health Insurance Portability and Accountability Act (HIPAA) Privacy Rule prohibits use or disclosure of protected health information except under specific authorizations, and de-identification under the Safe Harbor method requires the removal of 18 specified categories of identifier — names, geographic subdivisions, dates, contact numbers, record numbers, biometric identifiers, and the rest — before health data may be shared or repurposed (U.S. Department of Health and Human Services, 2012). Any organization training a clinical NLP model on scraped or repurposed patient data without express HIPAA authorization is operating outside the rule. But the same rule creates a second, subtler demand: the automated de-identification systems that redact those 18 categories at scale are themselves machine-learning models, and they must be trained and benchmarked on text with known, labeled identifier locations. That labeled text is precisely what cannot be safely drawn from real patient records, and what a construction-true synthetic de-identification corpus is built to supply. The California Consumer Privacy Act (CCPA), as amended by the California Privacy Rights Act (CPRA), adds opt-out and deletion-right requirements that are similarly incompatible with the architecture of frozen training corpora.

The cumulative effect is that compliance is no longer a feature that can be bolted onto a model after training. It is a constraint that determines whether the model can be lawfully trained at all in the relevant jurisdiction. A synthetic dataset, generated by a known model under documented controls, containing no real personal data, and accompanied by an explicit compliance coverage report, sidesteps the entire regulatory wall. The data subjects of GDPR, HIPAA, and CCPA are real people whose data was processed; synthetic data has no data

subjects. This is not a workaround. It is the only path that survives audit.

2.3 The Financial Exposure

The third force is economic, and it is the one most easily quantified. The most direct way to generate synthetic data without building infrastructure is to call a frontier generation API, which is priced per million tokens and billed separately for input and output. These rates establish the baseline against which any owned-infrastructure approach is measured. As of June 2026, OpenAI’s flagship GPT-5.5 is priced at \$2.50 per million input tokens and \$15.00 per million output tokens, with the more affordable GPT-5.4 at \$1.25 input and \$7.50 output (OpenAI, 2026). Anthropic’s Claude Opus 4.7 is priced at \$2.50 per million input tokens and \$12.50 per million output tokens, with the mid-tier Claude Sonnet 4.6 at \$1.50 input and \$7.50 output (Anthropic, 2026). Output tokens are priced five to six times higher than input tokens across the current generation of frontier models, and synthetic datasets generate substantial volumes of both: each record consists of an input prompt that is billed as input and a generated output that is billed as output.

These rates are reasonable for what they provide — on-demand access to a frontier model with no infrastructure to operate — but they compound under the workflow that produces a defensible dataset, and the raw output they return still requires the entire filtering, masking, evaluation, and audit layer documented in this paper before it is commercially deliverable. A single 100,000-record dataset generated against a frontier API runs into the hundreds of dollars, and scaled to one million records into the thousands (Section 9). More importantly, every figure recurs on every regeneration. The development of a defensible dataset is rarely a single pass — it includes threshold calibration, prompt-template iteration, placeholder-coverage expansion, regeneration after schema changes, and re-runs after rejection-metric review. The de-identification record type documented in this paper required exactly such an arc: a generation-quality defect, once exposed, took four successive fixes and many full regeneration cycles to resolve (Section 6). Under per-token API pricing, every one of those iterations is a billable event, and the meter does not stop between them. This is the cost structure that makes owned generation infrastructure worth building.

The alternative is owned generation infrastructure, and Veritrukt is built on two owned substrates rather than one. The local substrate, Veritrukt Studio, runs on a single NVIDIA RTX 4090 — current-generation, approximately \$1,800 retail — executing a 4-bit quantized generation model and a 4-bit quantized embedding model simultaneously on a residential workstation, where the marginal cost of operation is electricity. Drawing on a representative residential utility bill from Oklahoma City covering a 32-day billing period at 990 kWh of consumption (Oklahoma Gas & Electric, 2026), the blended all-in residential rate is approximately \$0.134 per kWh, inclusive of fuel cost adjustment, franchise fee, sales tax, and the Winter Event Securitization charge applicable to Oklahoma customers. The system’s power draw during generation was measured directly: total system draw is approximately 242 watts, or a cost of operation of approximately \$0.032 per hour (PBH Applied Systems LLC, 2026a). The cloud substrate, Veritrukt Cloud, moves generation to a Modal A10G GPU served by vLLM, at a metered cost of GPU-seconds rather than per-token model rental. Both substrates therefore impose a fixed capital or account cost that amortizes across an unbounded number of subsequent runs, leaving only a small marginal cost per run — a cost structure fundamentally different from the linear, per-token, per-regeneration charge of a frontier API. The full cost comparison, grounded in a measured 10,000-record production run on each substrate, is presented in Section 9.

The relevant point for the demand-creation argument is the structure rather than the absolute savings: cloud API generation imposes a linear per-token charge that scales without limit, while owned generation imposes a fixed-cost expense that amortizes across all subsequent runs. A publisher operating under cloud-API economics faces structural pressure to ship

the first plausible dataset and stop, because every iteration is a billable event. A publisher operating on amortized owned hardware can iterate until the rejection metrics, the calibration data, and the evaluation reports all converge on a defensible commercial claim. The two cost structures produce different products: one sized to the budget that generated it, the other sized to the quality standard the publisher chose to enforce. The de-identification yield-recovery arc documented in Section 6 — in which a false 95% yield was corrected to a true 35% and then rebuilt to 86.9% across four fixes — is the direct product of a cost structure that permitted that many iterations without a per-iteration penalty.

2.4 One Methodology, Two Substrates, Two Record Types

The cost analysis establishes the marginal-cost floor. A complete account of Veritrukt must also name the two structural dimensions along which the single methodology is deployed, because both bear on when and how the platform is used.

The first dimension is the substrate. Veritrukt Studio and Veritrukt Cloud run the identical methodology — the same generation model, the same embedding-driven quality gate, the same regulation-specific masking, the same evaluation pipeline — and differ only at the single stage where generation tokens are produced: in-process on the local RTX 4090 for Studio, and on a remote Modal A10G served by vLLM for Cloud. The substrate choice is an operational decision driven by the volume and timeline of a given engagement, not a difference in the data produced, and Section 9 documents a measured 10,000-record run on each substrate to quantify the tradeoff.

The second dimension is the record type. The same methodology emits two orthogonal products. An instruction fine-tuning record pairs an instruction with a quality-gated model response, filtered for instruction-output drift and masked for personal data; this is the record type whose hallucination gate and compliance architecture are documented in Sections 4 and 5. A construction-true de-identification record pairs a block of text with span-level labels marking the exact character offsets and types of every personal-data element the text contains, validated by a learned named-entity-recognition tier and a template-leak gate; this is the record type documented in Sections 5 and 6. The two record types share the generation, quality, masking, and evaluation infrastructure and diverge only in what they emit and how they are validated. The construction-true de-identification record is the newer and, in some respects, the more demanding of the two: because its labels are the product rather than an annotation applied after the fact, the correctness of every span offset is a first-class quality metric, and the discovery that a generation model silently emits fill-in-the-blank skeletons rather than completed records drove the most consequential engineering arc in the platform’s development.

2.5 The Layer That Is Missing

NVIDIA’s release of Nemotron-4 340B, together with its own disclosure that 98% of its alignment data is synthetic (Adler et al., 2024), and the parallel emergence of DeepSeek-R1 — an MIT-licensed open-source reasoning model trained on synthetic chains-of-thought generated by the model itself (DeepSeek-AI, 2025) — confirms that synthetic data is now the operating standard at the frontier. The category is validated. What is not validated, and what the open ecosystem has not yet built, is the production layer that sits between raw generation capacity and a commercially deployable, compliance-ready dataset deliverable.

That layer is a methodology, not a piece of hardware, and the central architectural claim of this paper is that the methodology is portable across generation substrates and applicable to more than one record type. The same quality and compliance components are applied identically whether generation runs on a local consumer GPU or on cloud infrastructure, and whether the emitted record is an instruction fine-tuning pair or a construction-true de-identification

span set. None of these components is provided by a foundation model or a base generation pipeline on its own:

A hallucination filter calibrated to the specific embedding distribution of the production pipeline, not inherited from a prototype or a literature default, and tuned per record type — a higher instruction-output floor for instruction fine-tuning, a lower floor for the richer completed text of de-identification records. Section 4 documents this filter.

A layered compliance filtration architecture that operates against named regulatory frameworks — GDPR, HIPAA, CCPA — combining tiered personal-information masking with severity-tiered keyword filters, and, for the de-identification record type, a template-leak gate that rejects fill-in-the-blank skeleton output that a naive gate would silently accept. Section 5 documents the masking architecture and the template-leak gate.

An automated evaluation pipeline that produces an attached, machine-readable report for every dataset shipped, covering rejection-reason breakdowns, scale-invariant diversity metrics, compliance coverage, and — for the de-identification record type — span-offset integrity and a learned-detector agreement score. Section 7 documents the evaluation pipeline.

A transparent residual-risk posture under which every delivered record has cleared the rejection gates, and residual measured rates are reported on the accepted set rather than suppressed. This distinction between gate-passed and provably perfect is the difference between a defensible quality claim and an indefensible one, and Sections 5 and 10 document it directly.

Because the methodology is independent of the substrate beneath it and of the record type above it, it can be deployed where the operational constraints of a given engagement dictate. Veritruct Studio runs it on a local consumer GPU at near-zero marginal cost; Veritruct Cloud runs it on cloud GPU infrastructure at higher throughput. The gates, the masking, the evaluation, and the audit are identical on both, and serve both record types. The remainder of this paper documents that layer, the two substrates that carry it, and the two record types it produces.

3 Architecture Overview

Veritruct is a single quality-gated methodology — instruction assembly, generation, semantic hallucination filtering, regulation-specific compliance masking, construction-true de-identification labeling, automated evaluation, and provenance-stamped delivery — implemented as a configuration-driven, modular Python monorepo and deployed across two complementary generation substrates. The defining architectural property is a strict separation between a shared core that contains the entire methodology and two thin substrate wrappers that differ only in where generation tokens are produced. Veritruct Studio runs generation in-process on a local NVIDIA RTX 4090, and Veritruct Cloud runs generation on a remote Modal A10G GPU (Modal Labs, 2026) served by vLLM (vLLM Project, 2026); both substrates generate with the Mistral-Nemo-Instruct-2407 model (Mistral AI, 2025). Both substrates import the same core, run the same quality gates, apply the same masking, emit the same two record types, and produce the same evaluation and delivery artifacts. This section documents that shared core, the two substrate wrappers, the two record types the methodology emits, and the runtime flow through which a record moves from request to attested, provenance-stamped output.

3.1 The Shared Core and the Two Substrates

The repository is organized as a single monorepo with three top-level Python packages and a set of shared asset and artifact directories (Figure 3.1). The heart of the system is `veritruct_core`, which contains the complete methodology as nine functional subpackages,

none of which is substrate-specific. The two substrate packages, `veritruct_studio` and `veritruct_cloud`, are deliberately thin: each contains only a backend adapter, a substrate-specific configuration file, a command-line interface, and a small set of operational scripts. The methodology lives in the core; the substrate packages exist only to bind that methodology to a particular generation backend.

Figure 3.1. *Veritruct repository architecture. The shared `veritruct_core` package (center) contains the entire methodology — configuration, generation orchestration, quality filtering, masking, de-identification, evaluation, and delivery. The two substrate packages, `veritruct_studio` (local RTX 4090) and `veritruct_cloud` (Modal A10G via vLLM), are thin backend wrappers that import the core unchanged. Shared `data/`, `runs/`, and `tests/` directories and a root configuration complete the tree. The single point of divergence between the substrates is the generation backend; every other stage is identical core code.*

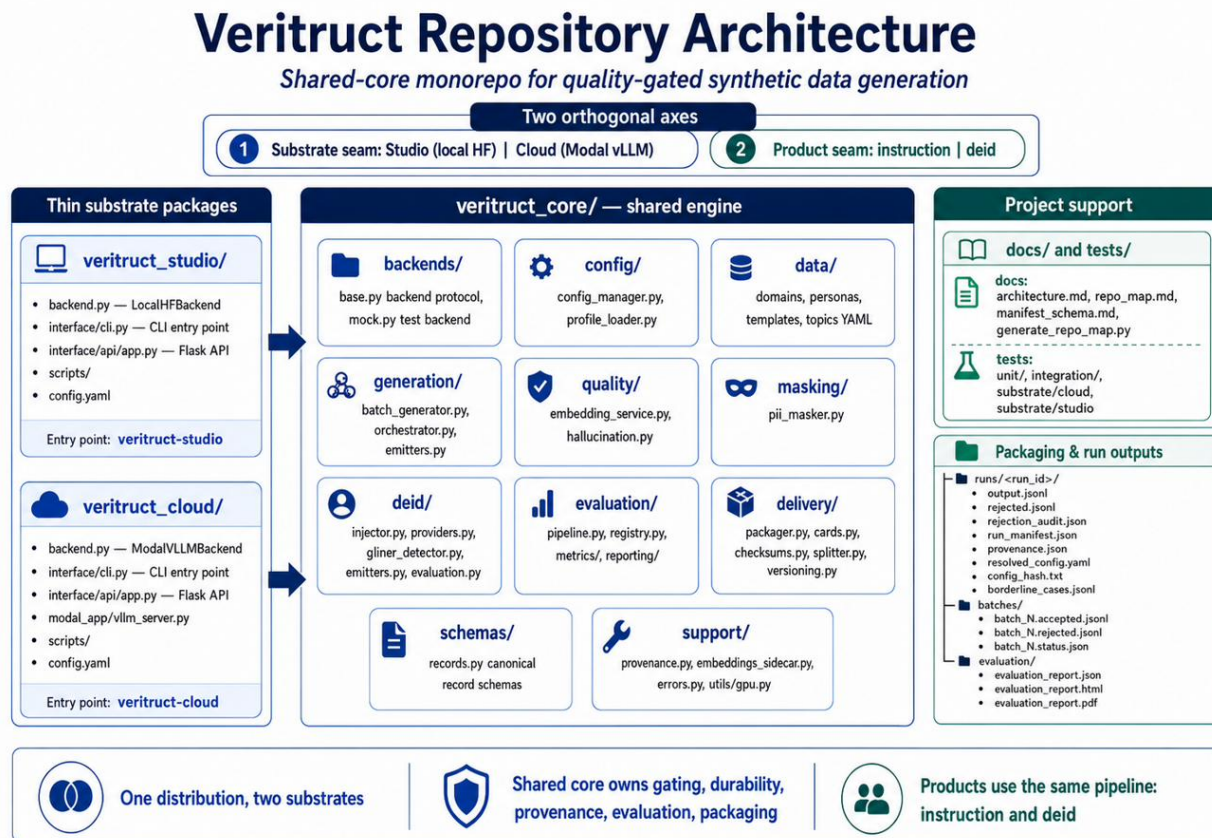


Figure 1: Figure 3.1 — Veritruct repository architecture

This separation is what makes either substrate auditable and what makes the “one methodology, two substrates” claim structural rather than aspirational. Because both substrate wrappers call into the identical `veritruct_core`, there is no opportunity for the quality gate, the masking, or the evaluation to differ between local and cloud generation — they are the same code objects, loaded from the same package, differing only in the backend handle passed to the generation stage. Configuration is declarative and lives in a per-substrate `config.yaml`; domain assets are version-controlled and static; runtime artifacts are written per-run under `runs/` and never modified after the fact; and executable code calls into configuration and assets but never the reverse. Every record produced can therefore be traced deterministically to the configuration, templates, core version, and substrate that produced it.

3.2 The Core Subpackages

The `veritrukt_core` package contains nine subpackages, each corresponding to a stage or concern of the methodology.

The `config` subpackage resolves runtime parameters through a two-layer model: `config_manager.py` loads the substrate’s master `config.yaml` and provides typed accessors, while `profile_loader.py` resolves a per-domain generation profile — a coherent bundle of model parameters, template selections, and quality-gate settings — merging per-domain settings with global configuration. This two-layer approach is what permits different domains, and the two record types, to operate with different thresholds without branching code paths.

The `generation` subpackage orchestrates the generative half of the pipeline. `orchestrator.py` is the entry point invoked by the interface layer and coordinates the full lifecycle of a run: profile resolution, the run loop, gating, retries, per-batch checkpointing, and manifest writing. `batch_generator.py` assembles instruction batches from the templated prompt libraries, applying domain weighting. `emitters.py` shapes the accepted output into the appropriate record type. The generation stage is the one place the substrates diverge internally: it either dispatches to the in-process model (Studio) or resolves a remote vLLM handle and dispatches batches over the network (Cloud), through a common backend interface.

The `backends` subpackage defines that common interface. `base.py` declares the abstract generation backend contract, and `mock.py` provides a deterministic test backend; the concrete Studio and Cloud backends live in their respective substrate packages and implement this contract. This is the seam along which the two substrates are interchangeable.

The `quality` subpackage is the first gate every generated output traverses, and it runs locally on both substrates. `embedding_service.py` wraps the Snowflake Arctic Embed L v2.0 model and computes instruction-output cosine similarity; `hallucination.py` implements the dual-signal gate logic, comparing each pair against the calibrated, record-type-specific threshold and routing the output onward or to the rejection log. Section 4 documents the gate.

The `masking` subpackage contains `pii_masker.py`, the entry point into the layered compliance filtration architecture — typed-placeholder PII substitution, an always-mask floor, severity-tiered keyword filters, and fidelity checks against GDPR, HIPAA, and CCPA. Section 5 documents it.

The `deid` subpackage is the machinery of the construction-true de-identification record type, and it is the largest single addition relative to a conventional instruction-only pipeline. `value_packet.py` pre-generates concrete, non-PII narrative values so the model completes records rather than emitting fill-in-the-blank skeletons; `injector.py` injects typed PII tokens while preserving exact character-span offsets; `providers.py` supplies the synthetic identifier values; `gliner_detector.py` runs the learned named-entity-recognition tier that independently validates the labeled spans; `emitters.py` shapes the span-labeled records; `evaluation.py` and `report_builder.py` compute and render the de-identification-specific metrics. Sections 5 and 6 document this subpackage in detail.

The `evaluation` subpackage operates after generation completes. `pipeline.py` drives the post-run evaluation against the accepted output set through a `registry.py` of metric modules, and produces the machine-readable and rendered reports. Section 7 documents the evaluation pipeline.

The `delivery` subpackage turns an evaluated run into a shippable, provenance-stamped deliverable. `packager.py` assembles the dataset package; `cards.py` generates the dataset card; `checksums.py` and `versioning.py` stamp integrity hashes and version identifiers; `splitter.py` produces train/validation/test partitions. Together with the top-level `provenance.py` and `embeddings_sidecar.py` modules, this subpackage is what makes a Veritrukt deliverable an attested artifact rather than a bare file.

The `schemas` subpackage defines the record contracts in `records.py`, so that both record types serialize to validated, versioned structures.

3.3 The Two Substrate Wrappers

The two substrate packages are intentionally minimal. `veritrukt_studio` contains `backend.py` — the concrete backend that loads the 4-bit quantized generation model into the RTX 4090’s memory and invokes it in-process — together with a `config.yaml` tuned to the local hardware, a `cli.py` interface, and operational scripts. `veritrukt_cloud` mirrors this structure but its `backend.py` resolves a handle to a deployed Modal vLLM service and dispatches batches to it, and it adds a `modal_app/vllm_server.py` that defines the remote A10G service and a `benchmark_batch_size.py` script that determines the optimal generation batch size for the deployed engine. Both substrates carry the same operational utilities — `mask_audit.py`, `clean_legacy_datasets.py`, `report_summary.py`, `run_gliner_eval.py`, and `verify_package.py` — because those utilities operate on core artifacts and are substrate-independent.

The critical property is that everything below the backend seam is shared. The embedding model that drives the quality gate runs locally on the RTX 4090 in both substrates — even in Veritrukt Cloud, where token generation is remote — so the quality signal stays on operator-controlled hardware regardless of where generation happens. The masking, the de-identification labeling and validation, the evaluation, and the delivery packaging are all `veritrukt_core` code executed locally in both deployments. Only the generation call crosses the substrate boundary.

3.4 The Two Record Types

The methodology emits two orthogonal record types from the same infrastructure, and the divergence between them is confined to the emitter and the validation path.

An instruction fine-tuning record is an instruction paired with a quality-gated model response. It is generated by the standard generation path, filtered by the hallucination gate for instruction-output drift, and masked for personal data by the compliance layer before delivery. Its quality is characterized by instruction-output cosine similarity, lexical diversity, and compliance coverage.

A construction-true de-identification record is a block of text paired with span-level labels marking the exact character offset, length, and type of every personal-data element the text contains. Its distinguishing property is that the labels are the product, not an annotation applied after the fact: because the synthetic values are injected into the text by `injector.py` at known offsets, the ground-truth span labels are correct by construction rather than by a fallible post-hoc tagging pass. This record type is validated by three mechanisms beyond the standard gate — the value-packet-first generation that prevents skeleton output, the learned GLiNER detector tier that independently confirms the labeled spans, and the template-leak gate that rejects fill-in-the-blank residue. Sections 5 and 6 document these mechanisms; the architectural point here is that they are additional validation applied to a record type emitted by the same core, not a separate pipeline.

3.5 Runtime Flow, Evaluation, and Deliverables

Figure 3.2 traces a run from interface invocation through generation, gating, evaluation, and delivery, showing where the two substrates diverge and where they converge. The flow is identical on both substrates except at the generation stage. Configuration and profile resolution occur once at the start of a run. The generation stage invokes the in-process model (Studio) or the remote Modal vLLM service (Cloud). Every stage downstream of generation —

the embedding-driven quality gate, the masking and de-identification validation, the accept-or-reject routing, the evaluation pipeline, and the delivery packaging — is `veritrukt_core` code that runs locally on both substrates.

Figure 3.2. *Veritrukt runtime flow, evaluation, and deliverables.* A run resolves configuration and profile once, then proceeds through the substrate-specific generation stage into the shared quality gate, masking and de-identification validation, and acceptance routing. Accepted records are written to `output.jsonl`, rejected records with their reasons to `rejected.jsonl`, and a rejection audit, run manifest, and embeddings sidecar are stamped alongside. The evaluation pipeline runs against the accepted corpus and emits a machine-readable and rendered report; the delivery layer packages the evaluated run into a provenance-stamped, checksummed, versioned deliverable with a dataset card.

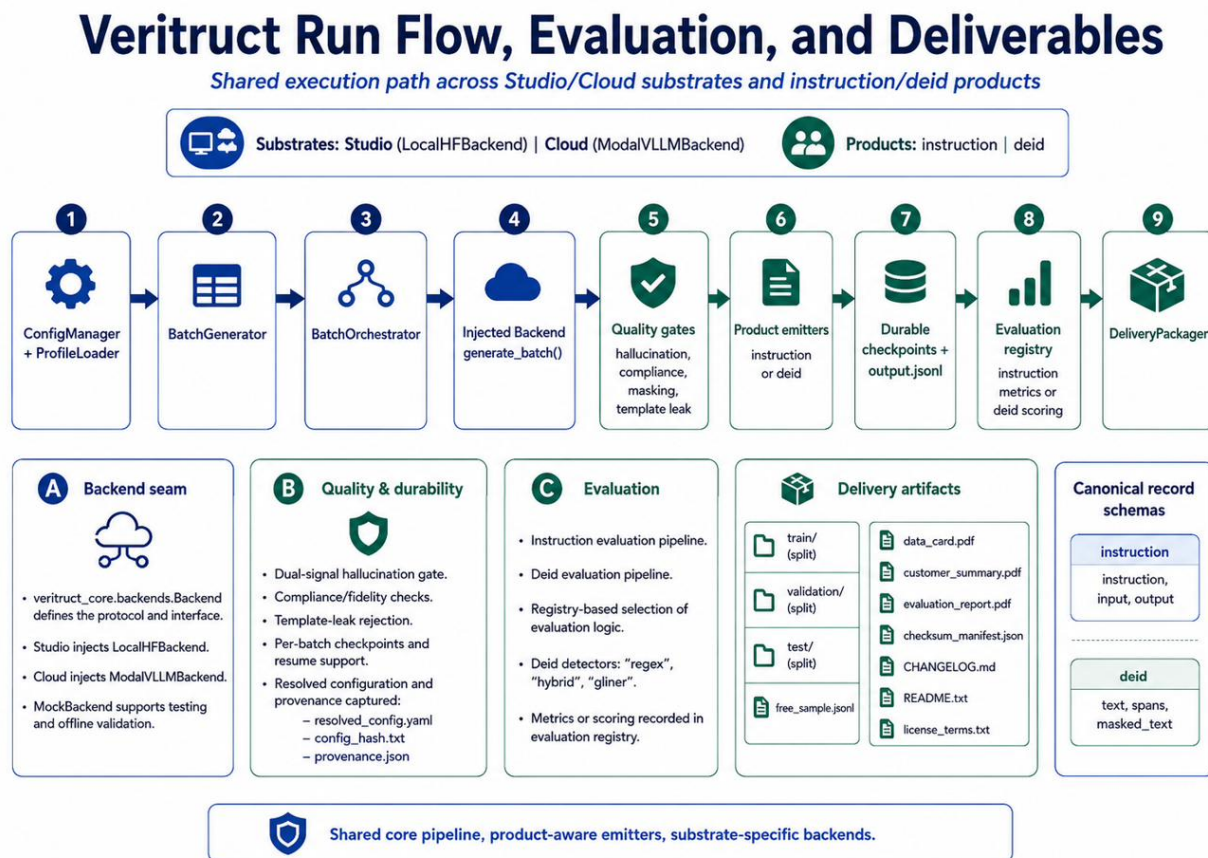


Figure 2: Figure 3.2 — Veritrukt runtime flow, evaluation, and deliverables

Every invocation of either substrate produces a complete, self-contained run record under `runs/`, written to a timestamped directory whose name encodes the substrate — `run_studio_YYYYMMDD_HHMMSS` or `run_cloud_YYYYMMDD_HHMMSS`. The directory contains the accepted records in `output.jsonl`, the rejected records with their originating instructions and rejection reasons in `rejected.jsonl`, a `rejection_audit.json` aggregating rejection counts by reason, a `run_manifest.json` recording the run's full metadata and provenance, an `embeddings.npz` sidecar holding the instruction-output embedding vectors, and an `evaluation/` subdirectory containing the machine-readable and rendered evaluation reports. The append-only discipline is structurally enforced: once a run completes, its artifacts are never rewritten, and reruns generate new timestamped directories. Veritrukt Cloud additionally checkpoints each batch's accepted and rejected records to disk as the batch finalizes, so that an interrupted

cloud run leaves its completed batches durably recoverable and independently evaluable.

The delivery layer is what converts a completed run into a commercial deliverable. The packager assembles the accepted records, the evaluation report, and a generated dataset card into a versioned package; checksums and versioning stamp integrity hashes and a version identifier; the provenance module records the generating model, configuration, core version, and substrate. The result is an attested artifact whose every claim — record counts, rejection reasons, diversity, compliance coverage, span integrity — is backed by a machine-readable report and a reproducible provenance trail. The remainder of this paper documents the subsystems this architecture composes: the hallucination gate (Section 4), the compliance and de-identification architecture (Section 5), the template-leak gate and yield recovery (Section 6), the evaluation pipeline (Section 7), the production results (Section 8), the economics of the two substrates (Section 9), the limitations (Section 10), and the reference appendix (Section 11).

4 The Hallucination Gate

The first quality gate every generated record traverses is the hallucination filter. It is the component most directly responsible for the central commercial claim of the methodology — that a delivered record actually answers the instruction it was generated against — and it is the platform’s primary technical differentiator. This section documents what the gate measures, the dual-signal decision rule that determines acceptance, the recalibration of the operating threshold that the de-identification record type forced, and the production evidence from a log-verified ten-thousand-record run.

4.1 Hallucination as Instruction-Output Drift

In a synthetic corpus, a hallucinated record is one whose output does not respond to its instruction. The failure is not factual error in the abstract sense — the pipeline does not adjudicate truth claims about the world — but semantic drift between the instruction that was issued and the text that was generated in reply. A record instructing the model to draft a clinical progress note, answered with a generic disclaimer that never addresses the patient encounter, is a hallucination in this operational sense: the output drifted off the instruction. For a buyer fine-tuning a model on the corpus, such records are worse than useless, because they teach the model to respond to a prompt with off-target content. Removing them is the difference between a dataset that improves a model and one that degrades it.

Detecting this drift at scale requires a measure of semantic correspondence between instruction and output that can be computed automatically for every record. The methodology uses embedding cosine similarity as that measure, computed by the local embedding service against a calibrated, domain-specific threshold.

4.2 The Dual-Signal Decision Rule

The gate is not a simple cosine floor. It is a two-signal decision, implemented in `veritrustruct_core.quality.hallucination`, in which a record is rejected as hallucinated only when both an embedding-similarity check and a keyword-overlap check fail simultaneously. This design is the single most important property of the gate, and it allows the filter to achieve high precision against off-topic drift without sacrificing recall on terse, on-topic, domain-specific outputs.

The first signal is embedding cosine similarity. The embedding service computes a vector for the instruction and for the output using the Snowflake Arctic Embed L v2.0 model (Snowflake, 2024; Yu et al., 2024), and takes the cosine similarity between the two. The similarity is

computed on the raw generated output, before the masking layer substitutes any personal-data spans, so the score reflects the model’s actual response. This raw-output cosine is the value persisted to the record as its quality signal, and the full vector is written to the run’s embeddings sidecar. The first signal fails when the similarity falls below the record’s domain threshold.

The second signal is a lightweight keyword-overlap check that serves as a semantic safety net. It confirms that the output is at minimum lexically anchored to the instruction’s subject, even where the embedding similarity is low.

The decision rule combines the two as a logical AND:

A record is flagged as hallucinated only if the cosine check fails **and** the keyword-overlap check also fails.

The consequence is that a low cosine score alone does not reject a record. If the output is lexically anchored to the instruction — if the keyword-overlap check passes — the record is accepted even when its cosine similarity sits below the domain threshold, on the reasoning that an output sharing the instruction’s substantive vocabulary is very likely on-topic despite an embedding score the model under-assigned. The cosine threshold is therefore a necessary but not sufficient condition for rejection. Each pair’s result is returned as a structured `HallucinationResult` recording the cosine score, both signal outcomes, the threshold applied, and whether the score fell within a ± 0.05 borderline window around the active threshold; borderline cases are logged to a per-run `borderline_cases.jsonl` for threshold-calibration review. This AND-design and the borderline-logging window are unchanged throughout the project; what changed is the cosine floor the first signal is measured against.

4.3 Recalibrating the Threshold: The 0.45 Unified Floor

The current committed configuration sets the cosine floor at 0.45, applied broadly. The Studio configuration uses 0.45 as both the default and every per-domain threshold. The Cloud configuration matches — 0.45 default and 0.45 for every domain — with a single documented exception: legal remains at 0.58, because legal outputs draw on more domain-specific vocabulary and warrant a higher floor to avoid false acceptances. The operative story is therefore a unified 0.45 floor with one legal=0.58 Cloud exception, not a per-record-type split.

That 0.45 landing point is the result of a recalibration driven by the de-identification work. The earlier regime used 0.55 as the default and 0.58 for the medical and legal domains — the correct floor for the output profile at the time. Once the de-identification generation fixes documented in Section 6 began producing fully completed records — documents populated with concrete synthetic values rather than fill-in-the-blank skeletons — those richer outputs scored measurably lower cosine-to-instruction. The reason is structural: a long, detailed, completed record diverges more from its short generating instruction than a sparse template does, so the very act of fixing the generation quality depressed the instruction-output similarity of the good records. Under the old 0.55/0.58 floor, a large fraction of these legitimate completed records were being false-rejected as hallucinations — empirically, the great majority of the hallucination-flagged de-identification records sat in the borderline 0.45–0.58 band and were, on inspection, genuinely good records. Lowering the floor to 0.45 recovered them while still rejecting the genuinely off-topic outputs, which score below 0.45 and continue to fail both signals.

Because the recalibration reflects a property of completed-record output in general — not something unique to a single record type — the 0.45 floor was adopted as the unified operative threshold across both substrates, rather than maintained as a de-identification-only override. The legal=0.58 Cloud exception is retained as the one case where higher domain-specific vocabulary justifies a stricter floor. The prior 0.55/0.58 regime is best understood

as historical calibration context: it was the correct floor for the earlier, sparser output profile and was revised downward once generation quality shifted toward completed records. A reader opening the committed configuration today will find 0.45 across the board, with `legal=0.58` on Cloud, consistent with this account. The thresholds are declared in configuration rather than hard-coded, so any future shift in the output profile can be met by re-reading the borderline-case log and adjusting the floor without a code change.

4.4 Production Validation

The gate’s behavior at scale is demonstrated by a log- and manifest-verified ten-thousand-record production run on Veritrukt Studio (`run_studio_20260629_212754`), a medical-domain de-identification run that generated 11,528 records and accepted 10,016, for a pass rate of 86.88%. Of the 1,512 rejections, the hallucination gate accounted for 1,049 — the largest single rejection category — with the remainder split across template-leak (452), bias (40), and fidelity (1) rejections. That the hallucination gate is the dominant rejection path on a completed-record de-identification run is exactly what the 0.45 recalibration anticipates: the gate is doing the bulk of the quality work, removing genuinely off-topic completions while the lowered floor admits the legitimately lower-cosine completed records.

This run is also the basis for a precise statement of what the gate does and does not establish. The 86.88% pass rate is a generation-time gate result, read directly from the run manifest and log; it is the fraction of generated records that cleared the hallucination, template-leak, bias, and fidelity gates during generation. It is not, in itself, a learned-detector score: the generation run proceeded from generation directly through the gates to rejection audit, manifest, and delivery, and the learned named-entity-recognition detector documented in Section 5 was exercised in a separate evaluation pass over this run’s delivered output rather than inline during generation. That separate detector evaluation is reported in Section 7, where the span-level detector metrics belong; the claim this section supports is narrower and fully backed by the manifest: the dual-signal gate, at the recalibrated 0.45 floor, accepted 86.88% of a ten-thousand-record medical de-identification run, with hallucination as the dominant and expected rejection reason.

4.5 Substrate Independence of the Gate

The hallucination gate is independent of the generation substrate. The embedding model that computes the cosine signal runs locally on the workstation in both deployments — even in Veritrukt Cloud, where token generation is offloaded to the remote Modal A10G service. The gate therefore evaluates Studio-generated and Cloud-generated records on identical local hardware, with the identical embedding model, the identical dual-signal rule, and the identical configuration structure. The only deliberate cross-substrate difference is the `legal=0.58` threshold retained on Cloud, a single documented configuration value rather than a difference in the gate itself. This is the hallucination-gate instance of the paper’s broader thesis: the methodology is portable across substrates because the quality layer is substrate-independent by construction, running as shared `veritrukt_core` code on operator-controlled hardware regardless of where tokens are produced. Section 5 documents how the gate’s per-record outputs feed into the compliance and de-identification layers, and Section 7 documents how they feed the evaluation pipeline and the attested report.

5 The Compliance and De-identification Architecture

The second quality layer every record traverses is compliance filtration, and it is the layer that most distinguishes the two record types. Where the hallucination gate of Section 4 asks whether an output answers its instruction, the compliance layer asks whether the output is

safe to deliver and, for the de-identification record type, whether its personal-data spans are labeled correctly. This section documents the layered masking architecture shared by both record types, the construction-true labeling that makes the de-identification record type's spans correct by construction, the value-packet generation and token injection that produce completed records with exact offsets, and the learned named-entity-recognition detector that independently validates those spans. The template-leak gate that governs de-identification acceptance, and the yield-recovery arc it exposed, are documented separately in Section 6.

5.1 Compliance as a Gate, Not a Disclaimer

The defining property of this layer is that compliance is enforced at generation time, against named regulatory frameworks, and verified on the delivered output — it is not a license disclaimer appended to a dataset card. The masker in `veritrustruct_core.masking.pii_masker` operates against an active regulatory profile — GDPR, HIPAA, or CCPA, selected in configuration — and applies framework-appropriate detection to every generated record before that record is eligible for delivery. Records that cannot be made compliant are not delivered; they are written to the run's rejection log with their reason. The delivered dataset is therefore clean by construction, and the cleanliness is a measured property of the output rather than an assertion about intent. The generator is allowed to generate naturally, and the compliance layer removes or masks what does not pass — the dataset is made clean by subtraction at the gate, not by coercion of the model.

5.2 Layered PII Masking

The `PIIMasker` applies domain-aware, typed-placeholder detection through a registry of compiled regular expressions, `_PII_PATTERNS`, covering the full identifier vocabulary — email, phone, national identifier, account number, credit card, IBAN, SWIFT/BIC, IP and MAC addresses, URL, license plate, order identifier, ZIP, date, and the medical code types ICD-10 and CPT. Each detected element is associated with a typed placeholder — `<EMAIL>`, `<IP_ADDRESS>`, `<NATIONAL_ID>`, `<CPT_CODE>`, and so on — so that the structure of the record is preserved while the sensitive value is represented as a labeled token.

Two properties make the masking robust rather than cosmetic. The first is an always-mask floor, `_ALWAYS_MASK`: a set of high-sensitivity identifier types is masked in every domain regardless of that domain's configured allowed-placeholder list, so a high-sensitivity value cannot pass merely because a domain's configuration did not anticipate it. The second is domain gating: outside the always-mask floor, the per-domain allowed and forbidden placeholder sets declared in configuration govern which types are recognized in a given domain — an `<IBAN>` is not treated as a valid placeholder in a medical record even if the model emits one, because the medical domain forbids it. This domain gating is the same rule applied throughout the de-identification path, and it is what keeps the two record types consistent with the domain profiles.

Beyond identifier masking, the compliance layer applies two severity-tiered keyword filters that operate on content rather than identifier patterns. The bias filter maintains a severe tier (`_BIAS_SEVERE`) whose matches reject a record outright in every domain, and a moderate tier (`_BIAS_MODERATE`) of terms that are sensitive but legitimately appear in professional discourse, which are handled with domain awareness so that on-topic professional content is not rejected for using the vocabulary its subject requires. The GDPR keyword filter is likewise tiered (`_GDPR_SEVERE` and `_GDPR_MODERATE`): the severe tier triggers rejection, while the moderate tier — the standard vocabulary of data-protection practice — is informational only and never gates delivery, because in a dataset whose purpose includes GDPR-relevant content the appearance of that terminology is the subject matter, not a violation.

5.3 Construction-True De-identification Labeling

The de-identification record type is a different primitive from the instruction fine-tuning record. It is a span-labeled pair: a block of text together with a set of labels marking the exact character offset, length, and type of every personal-data element the text contains. This is precisely the training and benchmarking substrate a de-identification model requires — text with known, labeled personal-data locations — and it is precisely what cannot be safely drawn from real records, because real records carry real identifiers.

The defensible core of the record type is how those labels are produced. A conventional synthetic de-identification corpus is generated first and then labeled by a tagging pass — a regex sweep or a model — which means the labels are only as good as the tagger and inherit its errors, exactly the errors a de-identification benchmark exists to measure. Veritrukt inverts this. The labels are not applied after the fact; they are correct by construction, because the personal-data values are injected into the text at positions the pipeline itself chooses and records. The values are generated, not collected; the offsets are known because the injector wrote them; and the ground-truth label set is therefore a byproduct of construction rather than the output of a fallible post-hoc tagging step. This is what “construction-true” names, and it is the property that lets a Veritrukt de-identification dataset serve as ground truth rather than merely as another tagged corpus.

5.4 Value-Packet Generation and Token Injection

Making construction-true labeling work at generation time required solving a problem that is invisible until the labels are the product: a general-purpose generation model, asked to produce a realistic completed record, does not reliably produce one. It produces a skeleton — a template studded with fill-in-the-blank stubs like [Your Name], [Signature], or [Phone Number] — because that is a statistically reasonable rendering of many document types in its training data. For an instruction record this is merely low quality; for a construction-true de-identification record it is fatal, because a skeleton has no concrete spans to label and no completed narrative for a detector to learn from. Section 6 documents the discovery of this failure mode and its consequences for yield; the mechanism that resolves it lives here.

The resolution has two parts, corresponding to two kinds of field in a record. The personal-data fields — the elements that will become labeled spans — are handled by token injection. The model is prompted to emit typed placeholder tokens (<EMAIL>, <CPT_CODE>, and the rest) at natural positions, and `veritrukt_core.deid.injector` then replaces each token with a concrete, format-shaped value drawn from `providers.py`, recording the exact character span of each replacement. The recorded spans are co-referential to the injected text, so the ground-truth labels are exact by construction. The injector gates injection by the same domain allowed-type policy as the masker — only types the domain permits are injected, and forbidden types are passed through rather than labeled — and only tokens with a dedicated value generator are injected, so a syntactic look-alike such as <OUTCOME_2> or <NOTE_1_CONTENT> is not mistaken for a personal-data placeholder.

The non-personal-data narrative fields — author names, signatures, phone numbers in prose, addresses, titles, organizations — are the fields the model was skeletonizing, and they are handled by the value packet. `veritrukt_core.deid.value_packet` pre-generates a packet of concrete, non-PII narrative values and the generation prompt hands them to the model with an instruction to use them verbatim, removing the model’s excuse to emit a blank. These values are deliberately not span-labeled — they are not personal data in the label vocabulary — and they exist only to make the surrounding document read as a completed record rather than a template. The division of labor is the key: personal data flows through <TOKEN> injection and becomes labeled spans with exact offsets, while non-PII narrative flows through the value packet and becomes completed prose, and together they produce a realistic completed record whose every personal-data span is correct by construction.

5.5 The Learned Detector Tier

The construction-true labels are correct by construction, but a de-identification dataset is more valuable if it also carries a measurement of how a realistic detector performs against those labels — a measurement of the very task the dataset exists to support. Veritrukt provides this through a learned named-entity-recognition tier that is selectable in configuration and layered on top of the regex detector.

The regex detector, `PIIMasker.detect_spans`, has near-perfect precision on the structured identifier types its patterns target — dates, emails, medical codes, network addresses, and the like — but by construction it cannot recognize free-text entities such as a person’s name or an arbitrary institution identifier, because those have no regular structure to match. The learned tier fills that gap. `GlinerDetector` wraps the `fastino/gliner2-large-v1` model (Zaratiana et al., 2024) behind the identical `detect_spans(text, domain)` contract as the regex detector, loading the model lazily on first use so that importing the module is cheap. It requests spans from the model with `include_spans=True`, producing character offsets that conform to the same span schema as the regex path — `{start, end, type, value, source}`, with `source="gliner"` — and it maps the model’s natural-language labels onto the platform’s typed placeholder vocabulary through a label schema. Critically, it is domain-gated exactly as the regex masker and injector are: it emits only types in the active domain’s allowed list, so a run is restricted to whatever the domain profile permits rather than the global type set.

The two detectors are combined by `HybridDetector`, which exposes the same `detect_spans` contract and is therefore a drop-in replacement. The combination rule is precedence, not blending: the regex detector runs first and wins every contested region — any learned span that overlaps a regex claim is discarded — so the regex detector’s high structured-type precision is preserved intact, and the learned tier contributes only in the regions the regex detector leaves unclaimed. The detector is selected through the evaluation configuration (`evaluation.detector: regex | hybrid | gliner`, defaulting to `regex`) and dispatched through a registry factory, `build_deid_detector`. The default regex detector imports nothing heavy and reflects the base product’s documented structured-type coverage; the hybrid detector is the premium tier that adds learned recovery of the free-text types. The learned tier is deliberately focused rather than maximal: the label schema covers the full identifier vocabulary, but the deployed hybrid path concentrates the learned detector on the free-text gap types it uniquely recovers — principally person names and institution identifiers — because an earlier thousand-record calibration run found that extending it to certain structured types the regex path already handles well (account numbers and SWIFT/BIC codes) lowered precision without adding recall, and those types were removed from the learned tier’s remit. The learned detector’s measured performance against the construction-true labels, and the important distinction between its strong micro-averaged score and its weaker macro-averaged score, are reported in Section 7.

5.6 The Record-Level Acceptance Model

The masking, keyword, injection, and detection mechanisms combine into a record-level acceptance model with three outcomes, applied uniformly across both record types. A record carrying a hard violation — raw personal data surviving the masker, a severe bias match, a severe GDPR match, a forbidden placeholder for its domain, or, for de-identification records, a template-leak failure — is rejected outright and written to `rejected.jsonl` with its reason; it is never delivered. A record whose only exposure is detectable and substitutable personal data has those spans handled — masked in place for the instruction record type, or injected and labeled for the de-identification record type — and is admitted. A record with no hard violation but a recoverable defect, such as a placeholder-density shortfall or, for de-identification records, a residual skeleton stub, is routed to a bounded regeneration or repair pass and re-gated against the identical checks. The consequence, made measurable by the production

results, is that the metrics reported on the delivered dataset are the residual that survived a working filter, and the records that did not survive are accounted for by reason in the rejection log. The template-leak gate that anchors de-identification acceptance, and the four-fix arc through which its exposure of true generation quality was recovered, are the subject of Section 6.

6 The Template-Leak Gate and Yield Recovery

The single most consequential engineering episode in Veritrukt’s development concerns the de-identification record type, and it is the clearest demonstration of why the measurement infrastructure is the product rather than an accessory to it. A generation-quality defect that a naive quality gate had been silently accepting was exposed when the gate was widened, collapsing an apparent 95% yield to a true 35%, and was then recovered to 86.9% across four successive fixes. This section documents the defect, the template-leak gate that exposed it, and the recovery arc.

6.1 The Skeleton Defect

A general-purpose generation model, asked to produce a completed record — a discharge summary, a nursing note, an audit finding — frequently does not produce one. It produces a skeleton: a template studded with fill-in-the-blank stubs where concrete content should be. The model emits [Your Name], [Signature], [Admission Date], [Sharp/Dull/Aching/Throbbing/Other], or <INSERT_PATIENT_IDS_HERE> in place of real values, because a blank form is a statistically reasonable rendering of many document types in its training data. For an instruction fine-tuning record this is merely low quality. For a construction-true de-identification record it is fatal: a skeleton has no completed narrative for a detector to learn from, and its bracketed stubs are not labeled personal-data spans — they are the absence of content. Measured across the de-identification output, the model was emitting skeletons in roughly 65% of records.

The defect was not initially visible, and the reason it was hidden is itself the central lesson. The early template-leak gate recognized only one skeleton style: angle-bracket stubs of the form <INSERT_X>. Square-bracket skeletons — [Your Name], [Insert Date], the slash-option and field-label styles — slipped straight through the gate and into the accepted output. Because those records passed every gate, they were counted as accepted, and the run reported an apparent yield of roughly 95%. That number was false. It measured not the fraction of records that were genuinely complete, but the fraction that lacked the one skeleton style the gate happened to check for. The gate was reporting the quality of its own coverage, not the quality of the data.

6.2 The Template-Leak Gate

The correction was to widen the gate to recognize every skeleton style the model actually produces. The current gate, implemented in `veritrukt_core.generation.orchestrator`, uses two compiled patterns. The keyword pattern, `_TEMPLATE_LEAK_KEYWORD_RE`, catches the angle-bracket family (<INSERT_URL>, <PATIENT_IDS_HERE>, any <..._HERE>) and the keyword-square family ([Insert ...], [Enter ...], [Your ...], [TBD], [Redacted], [Placeholder]). The structural pattern, `_TEMPLATE_LEAK_STRUCT_RE`, catches the slash-option family ([Sharp/Dull/Aching], [Independent/Assisted/Dependent]), the field-label family ([Admission Date], [Employee's Full Name]), and the bare clinical-field abbreviations ([DOB], [MRN], [SSN], [NPI]).

Widening a rejection gate carries two risks, and the gate is designed against both. First, it must not reject legitimate typed placeholders: the injector depends on <UPPER_SNAKE> tokens such as <EMAIL>, [DATE_1], or [CPT_CODE_2], and these are spared by requiring the structural branch to match a title-case or lower-case field name, so an all-caps typed placeholder does not trigger it. Second, it must not reject legitimate bracketed citations, which are title-case

like field labels; a citation guard, `_CITATION_RE`, excludes any structural match containing a four-digit year or “et al.” The result is a gate that rejects the full range of skeleton styles while sparing the two classes of bracketed text that legitimately appear in completed records.

When the widened gate was applied, the true state of generation quality became visible. The apparent 95% yield fell to approximately 35%. Nothing about the model’s output had changed; the gate had simply stopped counting skeletons as complete records. This was not a regression — it was the measurement finally reporting the truth, and it is the necessary precondition for fixing the underlying defect. A gate that reports a false 95% gives an engineer no signal to act on; a gate that reports a true 35% defines the problem precisely.

6.3 The Four-Fix Recovery

With the defect measurable, it was resolved through four successive fixes, each targeting a distinct cause and each moving the yield measurably.

The first fix was a document-type change. The initial de-identification formats were letters, memos, and consent forms — document types that, by their nature, end in signature blocks and fill-in sections, and that therefore invite the model to skeletonize. Replacing them with completed-record types that have no blank-form convention — electronic health record notes, case notes, audit findings — removed much of the model’s structural incentive to emit stubs. This raised true yield from approximately 35% to approximately 54%.

The second fix was the hallucination-threshold recalibration documented in Section 4. The richer, completed output that the pipeline was now producing scored measurably lower cosine-to-instruction than the sparse templates it replaced, so the old 0.55/0.58 floor was false-rejecting good completed records. Lowering the floor to 0.45 recovered the borderline band of legitimate records that had been misclassified as hallucinations.

The third fix was structural and is the one that resolved the root cause: value-packet-first generation, documented in Section 5.4. Rather than leaving the model to invent — and therefore skeletonize — the non-PII narrative fields, the pipeline pre-generates concrete non-PII values (names, dates, phone numbers, addresses) in code and hands them to the model with an instruction to use them verbatim, while personal data continues to flow through `<TOKEN>` injection that preserves exact span offsets. This removed the model’s remaining excuse to emit a blank, because every narrative field it might otherwise have stubbed now arrived pre-filled. Value-packet-first generation raised true yield to approximately 76.8%.

The fourth fix was a batched repair pass. A record whose only remaining defect is a residual skeleton stub — with no other hard violation, no hallucination, and no density shortfall — is not discarded. It is collected and sent, in a single batched request rather than a full regeneration, for a targeted rewrite that replaces the bracketed stubs while preserving the typed `<UPPER_SNAKE>` placeholders the injector depends on, and the repaired record is then re-gated through the identical acceptance checks so that it is held to the same standard as a first-pass record. This “Phase 2.5” repair recovered records that were otherwise complete, raising yield to approximately 83.2% on Veritrukt Studio and approximately 72.8% on Veritrukt Cloud.

The net effect of the four fixes is a validated ten-thousand-record medical de-identification run on Veritrukt Studio at an 86.88% pass rate — the run documented in Section 4, generating 11,528 records and accepting 10,016. The trajectory from a false 95% through a true 35% to a genuine 86.9% is summarized in Table 6.1.

Table 6.1. *The de-identification yield-recovery arc. The initial 95% was a false reading produced by a narrow gate that recognized only one skeleton style; widening the gate exposed the true 35% baseline, which four successive fixes then recovered to a validated 86.9%.*

Stage	True yield
Apparent yield (narrow gate, false reading)	~95%
Widened gate exposes true generation quality	~35%
Fix 1 — document-type swap to completed records	~54%
Fix 2 — hallucination threshold lowered to 0.45	borderline band recovered
Fix 3 — value-packet-first generation	~76.8%
Fix 4 — batched template-leak repair pass	83.2% Studio / 72.8% Cloud
Validated 10K medical run (Studio)	86.88%

6.4 Why This Is the Central Result

The template-leak episode is the strongest evidence for the paper’s recurring claim that the measurement infrastructure is itself the product. A pipeline whose gate recognized only one skeleton style would have shipped a de-identification dataset in which roughly two-thirds of records were blank forms, and would have reported that dataset as 95% clean — a claim that would have survived any check the pipeline itself was capable of running, because the pipeline was blind to the defect. The dataset would have been worse than useless for its purpose: a de-identification model trained on skeletons would learn to recognize bracketed stubs rather than real personal data.

What prevented that outcome was not a better generation model but a more honest gate. Widening the template-leak detection did not improve the data; it improved the measurement, and the improved measurement is what made the four fixes possible and verifiable. Each fix was adopted because a number moved in response to it, and the final 86.9% is a number the gate can now be trusted to report because the gate has been shown to catch the failure mode it once missed. This is also why the near-zero marginal cost of local development, documented in Section 9, matters in practice: the recovery arc consumed many full regeneration cycles across its four fixes, and a cost structure that penalized each iteration would have created pressure to ship the false 95% and stop. The honest gate and the affordable iteration are complementary; together they are what converts a generation model’s raw, skeleton-prone output into an attested, construction-true de-identification dataset.

7 The Evaluation Pipeline and the Attested Report

Every dataset Veritrukt delivers is accompanied by a machine-readable evaluation report generated by the same infrastructure that produced the data. This is the artifact that converts “the records passed the gates” into something a buyer can audit: a structured, reproducible account of what was generated, what was rejected and why, how diverse and well-formed the accepted records are, and — for the de-identification record type — how a realistic detector performs against the construction-true labels. This section documents the evaluation pipeline, the reconciliation step that guards against silent data loss, and the de-identification detector evaluation, including the important distinction between its strong micro-averaged score and its weaker macro-averaged score.

7.1 Evaluation as a Deliverable

The evaluation pipeline in `veritrukt_core.evaluation` runs against a completed run’s accepted output and composes a set of independent metric modules into a single `EvaluationReport`, serialized to `evaluation_report.json` as the canonical machine-readable form and rendered for human review. The entire accepted corpus is evaluated — the pipeline does not sample — so the report describes the dataset that ships, not a proxy for it. Each metric module is independent, so a failure in one does not prevent the rest of the report from being produced,

and the report is treated as a primary deliverable rather than an internal diagnostic: it is what makes the methodology’s quality claims checkable by a third party rather than asserted.

For the instruction fine-tuning record type, the pipeline computes five metric families. Diversity measures lexical variety, with a scale-invariant moving-average type-token ratio as its headline grade alongside hapax ratio and token-concentration measures, so that the diversity grade does not fall mechanically as corpus size grows. Quality characterizes well-formedness — output length distribution, token-length entropy, repetition rate, and the distribution of the stored instruction-output cosine similarity. Compliance reports the residual personal-data leak rate, placeholder coverage and density, and the GDPR-surface and bias-flag rates documented in Section 5. Yield analysis reconstructs the accept-reject economics of the run, reporting the overall yield and a per-reason breakdown across the rejection categories. Clustering performs an unsupervised semantic grouping of the accepted outputs to expose corpus-level failure modes — mode collapse or single-domain saturation — that per-record gates cannot see.

7.2 Reconciliation

Before any metric is reported, the pipeline reconciles independent accounts of the run: the count of records present in the accepted output, the count in the rejection log, and the counts recorded in the run manifest at generation time, including the per-domain distribution. A run whose accounts agree is marked reconciled; a disagreement — a truncated file, a checkpoint that did not flush, a crash mid-write — raises a warning in the report. This check makes the integrity of the artifact itself, not merely the quality of its contents, part of the attested report. It is what allows a delivered Veritrukt dataset to state its own record counts with confidence, and it is the mechanism by which the append-only run discipline described in Section 3 is verified rather than merely asserted.

7.3 De-identification Detector Evaluation

The de-identification record type carries an evaluation the instruction record type does not: a measurement of how a realistic named-entity detector performs against the construction-true labels. This is a dual-use artifact. Because the labels are correct by construction, the detector’s scores grade the detector rather than the labels — a low recall on a given type means the detector missed spans that are known to be present, not that the labels are uncertain. The evaluation, implemented in `veritrukt_core.deid.evaluation`, matches each predicted span against the ground-truth spans by character-range overlap and tabulates per-type true positives, false positives, and false negatives, from which per-type and averaged precision, recall, and F1 are computed.

The detector evaluation reported here was produced by a hybrid regex-plus-GLiNER2 detector pass over the delivered output of the ten-thousand-record medical run documented in Sections 4 and 6 (`run_studio_20260629_212754, detector hybrid(regex+gliner2)`), scored against 49,344 construction-true ground-truth spans. This detector pass is separate from the generation run — it evaluates the delivered dataset rather than running inline during generation — and it is the source of the detector figures below.

The headline result is a micro-averaged F1 of 0.964, with micro-averaged precision of 0.986 and recall of 0.944. The micro-average pools true positives, false positives, and false negatives across all spans before computing the ratio, so it weights each of the 49,344 spans equally and is the appropriate summary for a dataset dominated by a handful of high-frequency identifier types. On the types that carry meaningful support in a medical de-identification corpus, the detector performs strongly and consistently (Figure 7.1): EMAIL at F1 1.000, DATE at 0.990, ICD-10 code at 0.972, national identifier at 0.963, and CPT code at 0.957, with URL at 1.000

on its smaller support. These five structured and free-text types account for essentially all of the corpus’s spans, and the detector clears F1 0.95 on every one of them.

Figure 7.1. *Per-type detector F1 on the 10,000-record medical de-identification run, for the types carrying meaningful support (≥ 50 ground-truth spans). The hybrid regex-plus-GLiNER2 detector clears F1 0.95 on every high-support type; the dashed line marks the micro-averaged F1 of 0.964 across all 49,344 spans. Support counts are labeled on each bar.*

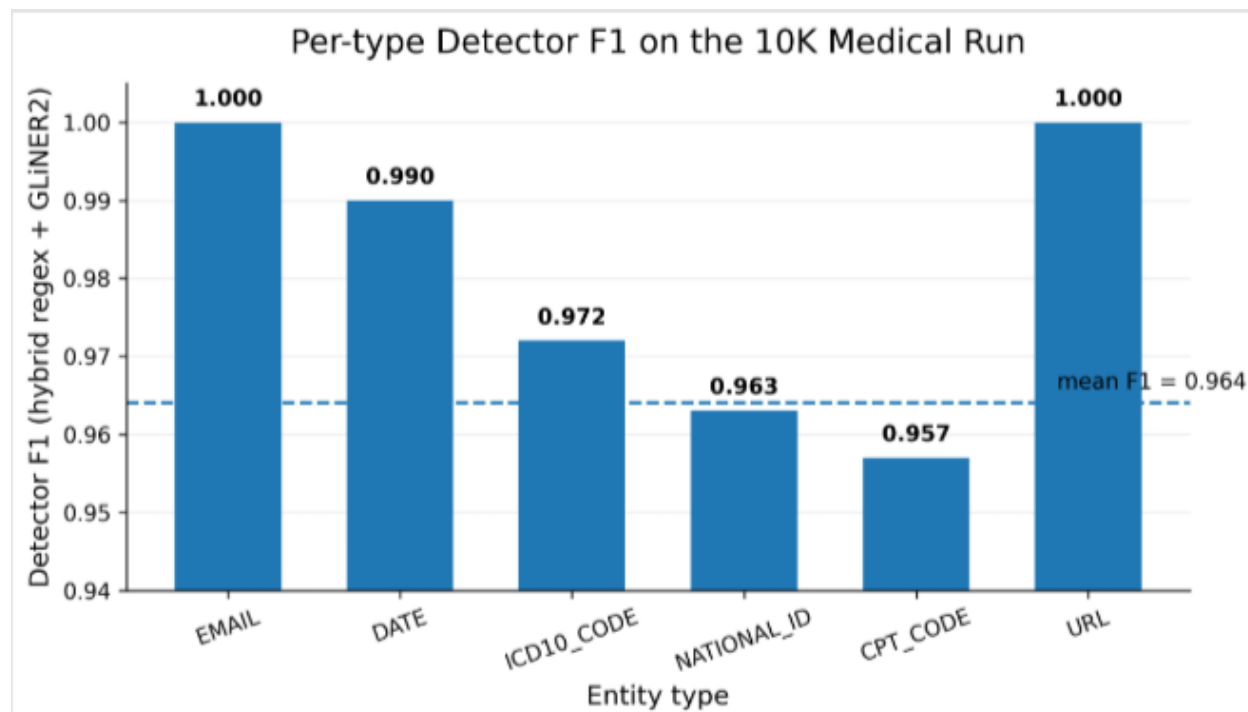


Figure 3: Figure 7.1 — Per-type detector F1, 10K medical run

Honesty about the averaging is essential here, because the same report carries a macro-averaged F1 of 0.588, far below the micro-averaged 0.964, and the gap must be explained rather than buried. The macro-average computes F1 for each type independently and then averages the per-type figures with equal weight regardless of support. The report includes several types with near-zero support — CLAIM_ID with a single ground-truth span, MODIFIER with three, and CREDIT_CARD and PHONE with none present in this medical corpus but a small number of spurious detector predictions. Each of these scores F1 0.000, and because the macro-average weights a one-span type identically to a twenty-thousand-span type, those empty and near-empty types pull the macro figure down to 0.588. The macro-average is therefore not a measure of detector quality on this dataset; it is an artifact of averaging strong performance on the types that are present with zero performance on types that are effectively absent. The micro-averaged 0.964 is the figure that reflects detector performance on the data that actually exists, and the macro-averaged 0.588 is reported alongside it in the interest of not overstating the result — a reader who saw only the micro-average would not know that four low-support types exist, and a reader who saw only the macro-average would badly underestimate the detector on the types that matter. Section 10 addresses the low-support types and the value-validity figure directly as limitations.

The same evaluation reports a value-validity tally: of the 49,344 injected span values, 42,055 were validated as well-formed for their type and 7,289 were not. This is a measure of the format fidelity of the injected synthetic values rather than of detector accuracy, and it is discussed with the limitations in Section 10.

7.4 The Detector Evaluation Is Configurable and Substrate-Independent

The detector used for evaluation is selected in configuration — `regex`, `hybrid`, or `gliner` — and dispatched through the registry factory documented in Section 5, so the same run’s output can be scored under the base `regex` detector or the premium `hybrid` detector without regenerating the data. Like every other stage below the generation seam, the evaluation runs as shared `veritrukt_core` code on local hardware regardless of which substrate generated the records, so a Studio-generated and a Cloud-generated de-identification dataset are evaluated by the identical pipeline against the identical construction-true criterion. The evaluation pipeline does not merely describe the dataset; together with the template-leak gate of Section 6, it is the mechanism by which the methodology’s de-identification quality is made measurable, and by which the difference between a genuinely completed, correctly-labeled record and a skeleton is turned into a number a buyer can inspect.

8 Results

The preceding sections documented each subsystem of the methodology against production data: the hallucination gate in Section 4, the compliance and de-identification architecture in Section 5, the template-leak gate and yield recovery in Section 6, and the evaluation pipeline in Section 7. This section consolidates the results, presenting the two ten-thousand-record de-identification runs — one on each substrate — as complete runs, and stating what they collectively establish about the methodology.

8.1 The Two Production Runs

Two log- and manifest-verified ten-thousand-record medical de-identification runs anchor the results, one on each substrate. The Studio run, `run_studio_20260629_212754`, generated 11,528 records and accepted 10,016 for a manifest-reported yield of 86.88%; its reconciliation confirms 10,016 accepted record identifiers against a single medical domain, and its labeling mode is recorded as `construction-true`. The Cloud run, `run_cloud_20260630_160703`, generated 11,776 records and accepted 10,064 for a yield of 85.46%. Both runs targeted 10,000 delivered records and retained a small surplus above target, and both were reconciled — the accepted and rejected counts agree with the manifest.

Table 8.1 consolidates the two runs. The figures are drawn from each run’s manifest and the run cost record.

Table 8.1. Consolidated results for the two ten-thousand-record medical de-identification runs. Yield and rejection figures are from each run’s manifest; cost and time figures are from the run cost record and are discussed in full in Section 9.

Metric	Studio 10K (RTX 4090)	Cloud 10K (A10G)
Run ID	<code>run_studio_20260629_212754</code>	<code>run_cloud_20260630_160703</code>
Record type / labeling	de-identification / construction-true	de-identification / construction-true
Domain	medical	medical
Generated (attempted)	11,528	11,776
Accepted (delivered)	10,016	10,064
Yield	86.88%	85.46%
Hallucination rejections	1,049	1,031
Template-leak rejections	452	680
Bias rejections	40	54

Metric	Studio 10K (RTX 4090)	Cloud 10K (A10G)
Fidelity rejections	1	2
Wall-clock time	~17.84 hours	~3.38 hours
Cost	~\$0.58 (electricity)	\$3.60 (Modal credits)

8.2 Substrate Behavior and Rejection Composition

The two runs behave consistently, which is the result the two-substrate thesis requires: the same methodology, applied through two different generation backends, produces de-identification datasets of comparable yield and composition. Yield is within one and a half percentage points across the substrates — 86.88% on Studio, 85.46% on Cloud — and in both runs the hallucination gate is the dominant rejection path, followed by template-leak, then bias, then fidelity (Figure 8.1). This ordering is exactly what the recalibrated 0.45 floor and the widened template-leak gate of Sections 4 and 6 predict: the two gates that do the most quality work on completed de-identification records are hallucination and template-leak, and they account for the overwhelming majority of rejections on both substrates.

Figure 8.1. *The two ten-thousand-record de-identification runs. Left: rejection composition on each substrate, showing hallucination and template-leak as the two dominant rejection paths and near-identical yield (86.9% Studio, 85.5% Cloud). Right: wall-clock time and cost at 10,000 records — the Cloud substrate delivers roughly 5.3 times faster at a still-trivial absolute cost, the tradeoff quantified in Section 9.*

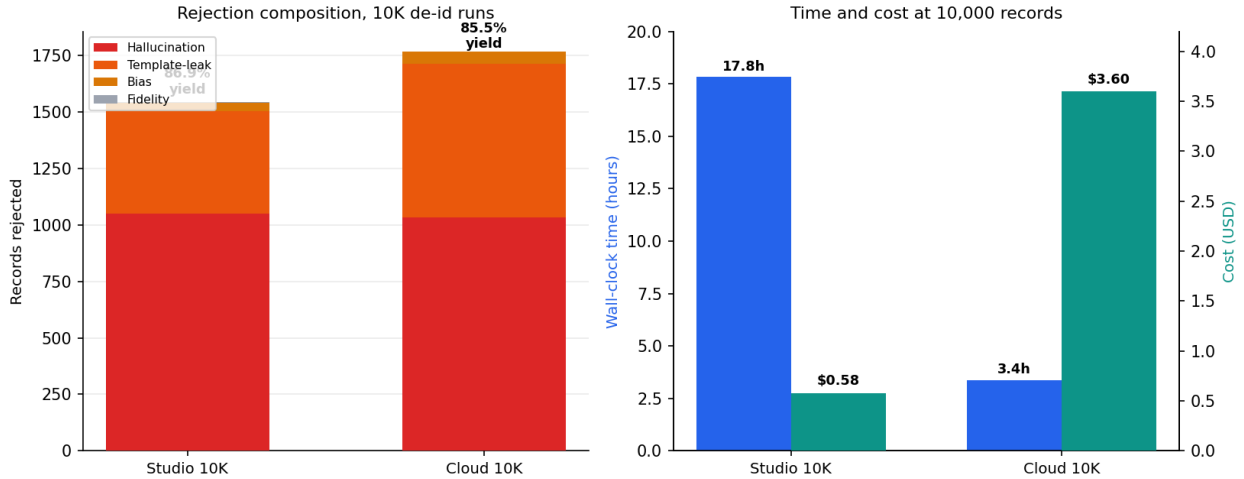


Figure 4: Figure 8.1 — Substrate comparison at 10,000 records

The modest differences between the runs are differences in rejection composition, not in quality. The Cloud run rejected somewhat more records to the template-leak gate (680 versus 452) and slightly more to bias (54 versus 40), which lowered its yield by about 1.4 percentage points relative to Studio. These are the compliance and quality gates doing their work on that run’s generated output, not a deficiency of the substrate; the hallucination-rejection counts are nearly identical across the two runs (1,031 versus 1,049), confirming that the gate that most directly measures instruction-output fidelity behaves the same regardless of where tokens were generated. The consistency of the hallucination-rejection count across substrates is the run-level echo of the substrate-independence property established in Section 4.5: the quality gate is the same local code evaluating both runs.

8.3 De-identification Label and Detector Quality

The headline quality result for the de-identification record type is the detector evaluation documented in Section 7, computed against the Studio run’s 49,344 construction-true ground-truth spans: a micro-averaged detector F1 of 0.964, with every meaningful-support identifier type — email, date, ICD-10 code, national identifier, and CPT code — clearing an F1 of 0.95. Because the ground-truth labels are correct by construction, this figure grades the detector against known-correct spans rather than against a fallible tagging pass, and it establishes that a realistic hybrid detector recovers the personal-data spans in the delivered dataset at high fidelity on every type that carries meaningful frequency in a medical corpus. The macro-averaged F1 of 0.588, as Section 7 explains and Section 10 revisits, is an artifact of low-support types rather than a measure of detector quality on the data that exists.

The value-validity tally from the same evaluation — 42,055 of 49,344 injected span values validated as well-formed, against 7,289 that were not — is a measure of the format fidelity of the injected synthetic values and is the clearest single target for the next round of improvement. It is reported here rather than omitted, consistent with the residual-risk posture of the methodology, and is discussed as a limitation in Section 10.

8.4 The Instruction Fine-Tuning Record Type at Production Scale

The two ten-thousand-record runs above characterize the de-identification record type. The instruction fine-tuning record type — the platform’s other first-class product, in which an instruction is paired with a quality-gated response — was validated by a separate production run on Veritrust Studio (`run_studio_20260705_203339`), a thousand-record run spanning all seven domains rather than the single medical domain of the de-identification runs. It generated 1,122 records and accepted 1,014, for a manifest-reported yield of 90.37%, and it reconciled — the accepted count and the seven-domain distribution agree with the manifest.

The instruction run is evaluated by the five-metric instruction pipeline of Section 7 rather than by a span detector, and its results establish that the instruction product meets the same quality bar as the de-identification product. Lexical diversity is strong: a scale-invariant MATTR of 0.790 over a vocabulary of 7,009 types, with a near-duplicate rate of zero. Compliance is clean: a residual PII leak rate of 0.0% from the independent scanner, full placeholder coverage, a bias-flag rate of 0.3%, and zero echo contamination. The rejection composition mirrors the de-identification runs in character — of 108 rejections, template-leak (63) and hallucination (45) are again the two dominant paths, with only small numbers of fidelity, GDPR, and bias rejections — confirming that the same gates do the same work regardless of record type. Per-domain yield is even across the seven domains, ranging from 87.6% in GDPR to 93.6% in privacy around the 90.4% overall (Figure 8.2, left). The instruction record type is therefore not a secondary capability but a fully-validated product: generated through the identical pipeline, gated by the identical quality and compliance layers, and shipped with the identical attested evaluation report as the de-identification record type.

8.5 The De-identification Detector Across the Full Type Vocabulary

The ten-thousand-record detector evaluation of Section 8.3 was medical-only, which — as Section 10 notes — left several identifier types with little or no support and untested. A second de-identification run on Veritrust Cloud (`run_cloud_20260706_000447`), a thousand-record run spanning all seven domains, exercises the detector across the full type vocabulary and directly addresses that gap. It generated 1,250 records and accepted 1,053 for a yield of 84.24%, reconciled across all seven domains, and its delivered output was scored by the same hybrid regex-plus-GLiNER2 detector against 2,452 construction-true ground-truth spans.

The all-domain detector result is a micro-averaged F1 of 0.905 (precision 0.836, recall 0.986)

and — more informatively than the medical-only run — a macro-averaged F1 of 0.811, far above the 0.588 of the medical run precisely because the low-support types are now populated. Across seventeen types with real support, the detector clears an F1 of 0.96 on fifteen of them, including every structured identifier type — email, order identifier, IP and MAC addresses, CPT and ICD-10 codes, national identifier, IBAN, license plate, credit card, date, URL, ZIP (Figure 8.2, right). The two types that score low are the free-text entities the learned tier is uniquely responsible for: STUDENT_NAME (F1 0.104) and SCHOOL_ID (F1 0.245), where the detector’s recall is high (0.957 on names) but its precision is low, meaning it finds the entities but over-predicts their boundaries. This is a genuine, honestly-reported weakness in the free-text tier rather than an averaging artifact, and because it is now measured across a populated type it is actionable in a way the medical-only run’s empty types were not. The value-validity tally on this run — 2,246 of 2,452 injected values well-formed, or 8.4% invalid — is materially better than the medical run’s 14.8%, indicating the injected-value fidelity improves outside the code-heavy medical domain.

Figure 8.2. The two thousand-record production runs. Left: per-domain yield for the instruction fine-tuning run (*run_studio_20260705_203339*) across all seven domains, even around a 90.4% overall. Right: per-type detector F1 for the all-domain de-identification run (*run_cloud_20260706_000447*), which exercises the full identifier vocabulary — fifteen of seventeen supported types clear F1 0.96, while the free-text STUDENT_NAME and SCHOOL_ID types (orange, red) show the learned tier’s high-recall, low-precision boundary weakness.

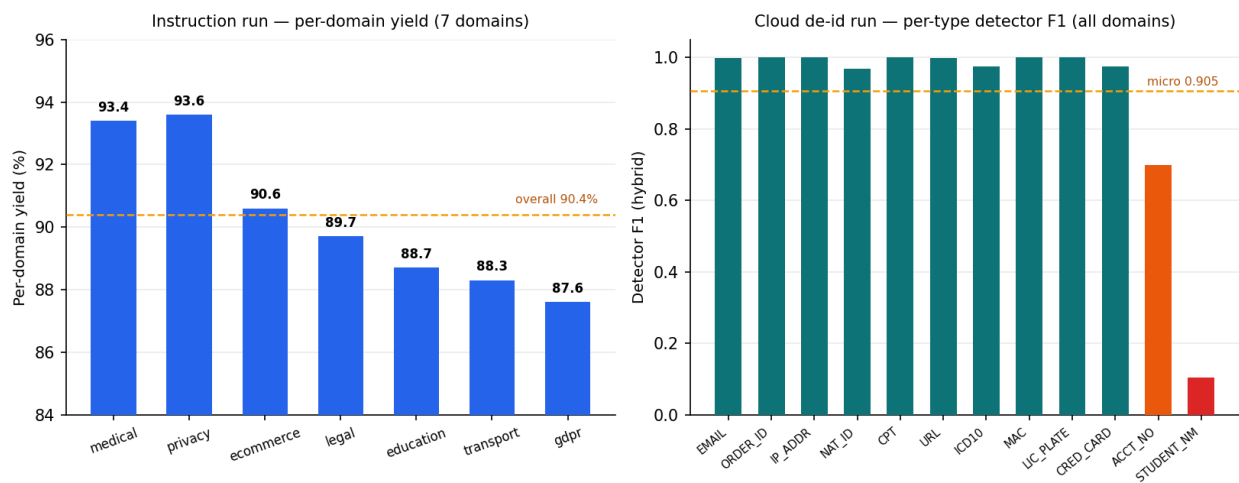


Figure 5: Figure 8.2 — The two thousand-record runs

8.6 What the Results Establish

Taken together, the four runs establish three things. First, the methodology produces both record types at production scale under a documented, auditable process: construction-true de-identification datasets with yields near 85% and detector micro-F1 scores of 0.905 to 0.964 against known-correct labels, and an instruction fine-tuning dataset at 90.4% yield with MATTR 0.790 and a zero residual PII leak rate. Second, those results hold across both substrates and both record types: Studio and Cloud produce datasets of comparable yield and identical rejection ordering, and the instruction and de-identification products pass through the same gates with the same composition, so the substrate and the record type are operational choices rather than quality determinants, exactly as the two-substrate, two-record-type thesis claims. Third, the results are reported with their limitations visible rather than hidden — yields are stated against the true, widened template-leak gate rather than the false 95% a narrow gate once reported; the detector’s micro-average is presented alongside its

macro-average with both the low-support artifact and the genuine free-text-precision weakness explained; and the value-validity shortfall is named as the next improvement target. The methodology does not claim perfection. It claims measured, reconciled, attested datasets of both record types, produced under a documented process on either of two substrates, and the four production runs substantiate that claim.

9 Throughput and Economics

The two production runs of Section 8 were generated on two substrates with very different throughput and cost profiles, and because both runs targeted the same ten-thousand-record deliverable, they constitute a direct, measured comparison rather than a projection. This section documents that comparison, draws out the cost structure that distinguishes owned generation from metered API rental, states the operational rule by which the substrate is chosen for a given engagement, and closes on what the attested-deliverable methodology is worth beyond the cost of generating tokens. The analysis treats frontier generation APIs as a neutral cost baseline throughout.

9.1 The Measured Ten-Thousand-Record Comparison

The two runs deliver the same product at the same scale, so their throughput and cost are directly comparable. The Studio run completed 10,016 accepted records in approximately 17.84 hours on the local RTX 4090, a throughput of roughly 561 accepted records per hour, at a marginal cost of electricity alone. At the measured system draw of approximately 242 watts and the blended residential rate of \$0.134 per kilowatt-hour established in Section 2.3, the run consumed approximately 4.32 kilowatt-hours and cost approximately \$0.58 — about six cents per thousand records (PBH Applied Systems LLC, 2026a).

The Cloud run completed 10,064 accepted records in approximately 3.38 hours on the Modal A10G, a throughput of roughly 2,978 accepted records per hour, at a metered cost of GPU-seconds. The run’s cost decomposes into \$3.17 of A10G GPU time, \$0.17 of CPU, and \$0.05 of memory — \$3.39 of compute — with the billed credit span of \$3.60 (a balance decline from \$18.42 to \$14.82) reflecting an additional roughly \$0.21 of container idle-tail before scaledown (PBH Applied Systems LLC, 2026b). At the billed figure, the Cloud run cost approximately 36 cents per thousand records.

The head-to-head is therefore clean and measured, not projected: the Cloud substrate delivered the same ten-thousand-record de-identification dataset roughly 5.3 times faster than the local substrate, at roughly 6.2 times the cost. Both figures matter, and both are small in absolute terms. The speed multiple is the operational advantage of the Cloud substrate; the cost multiple is its operational price; and the fact that the more expensive substrate still cost \$3.60 for ten thousand fully-gated, construction-true, evaluated records is the point that frames the entire economic argument. Neither substrate is expensive. The choice between them is a choice between “essentially free but slower” and “still very cheap but much faster,” and Section 9.3 gives the rule for making it.

9.2 Owned Infrastructure Versus Metered Rental

The measured figures are best understood not as two points on one cost curve but as a difference in cost structure, and that difference is the reason both substrates are owned rather than rented from a per-token API. Per-token API generation imposes a linear variable cost that scales without limit and recurs identically on every run and every regeneration: every record incurs a charge proportional to the tokens it consumes, every time the work is repeated. Both Veritrukt substrates instead impose a fixed cost that amortizes across an unbounded number of subsequent runs — the one-time purchase of the RTX 4090 for Studio, the Modal account

for Cloud — leaving only a small marginal cost per run, which is electricity locally and metered GPU-seconds in the cloud.

The magnitude of the gap is set by the frontier-API baseline established in Section 2.3. Against the per-token rates of the current frontier and mid-tier models and a representative record token profile, generating ten thousand records of raw output on a frontier API runs into the tens of dollars, and the same volume on a mid-tier API into the low tens — before any of the filtering, masking, evaluation, and audit that turns raw tokens into a deliverable. The Studio run’s approximately \$0.58 and the Cloud run’s \$3.60 are the fully-loaded generation costs of an attested deliverable on owned infrastructure, and they sit an order of magnitude or more below the raw-token cost of the metered alternative. More consequential than any single-run ratio is that the API cost recurs on every regeneration while the owned cost is paid once and amortized. The de-identification yield-recovery arc of Section 6 — four fixes across many full regeneration cycles — is the concrete illustration: under owned infrastructure the marginal cost of each of those iterations was cents to a few dollars, whereas under per-token API pricing each iteration would have been a fresh billable event, and the cumulative meter across the whole arc would have created pressure to ship the false 95% and stop.

This is stated as an economic fact rather than a competitive claim. Per-token APIs price on-demand access to a frontier model with no infrastructure to operate, which is real value for many uses. The narrow point is that for the repeated, iterative, high-volume generation that a quality-gated synthetic-data methodology entails, a fixed-cost-plus-low-marginal structure and a purely linear structure produce materially different total costs, and Veritrukt is built on the former across both of its substrates.

9.3 The Substrate Decision

Because Section 8 established that the two substrates produce de-identification datasets of comparable yield and identical rejection ordering, the quality of the output does not enter the substrate decision. The decision is therefore a clean operational tradeoff between the two measured quantities of Section 9.1: throughput and cost.

The rule follows directly. For low-volume deliverables, or deliverables on flexible timelines, the Studio substrate is the natural choice: its marginal cost is the lowest available — about six cents per thousand records — and its throughput of roughly 561 records per hour is more than adequate when wall-clock time is not the binding constraint. For high-volume deliverables under fixed deadlines, the Studio substrate’s throughput becomes the binding constraint — the ten-thousand-record run took the better part of a day — and the Cloud substrate is the natural choice, trading a still-trivial absolute cost increase (from cents to a few dollars) for a roughly five-fold reduction in delivery time. The crossover is a time crossover, not a cost crossover: Studio is cheaper per record at every volume, so the question for any engagement is simply whether its throughput meets the deadline, and if it does not, the Cloud substrate meets the deadline at a cost that remains a negligible fraction of the engagement’s value.

The Cloud substrate additionally makes the GPU tier itself an operational variable. Because Veritrukt Cloud selects its Modal GPU through a single configuration value, the rented tier can be matched to the deliverable’s volume and deadline without any change to the methodology, so the operator provisions a larger or faster GPU for a more demanding engagement and a smaller one for a routine one, paying only for the tier each engagement requires. The substrate decision is thus reducible to a single operational question per engagement — can the deadline be met at the available throughput, and if not, what provisioning meets it — with quality removed from the equation entirely.

9.4 The Value of Attestation

The cost analysis to this point concerns the price of generating and gating tokens. The larger economic implication concerns what is produced. A raw generation API, on either a rental or an owned substrate, produces tokens; the Veritrukt methodology produces an attested deliverable — for the de-identification record type, a construction-true, span-labeled dataset accompanied by a machine-readable evaluation report covering rejection reasons by category, a reconciliation of the run’s own counts, and a detector evaluation scored against known-correct labels. The difference between those two outputs is the entire production layer documented in Sections 4 through 7.

The economic value of that layer is realized at the point of use. A buyer training or benchmarking a clinical de-identification model cannot use raw generated tokens directly: the text must carry correct span labels, the labels must be trustworthy enough to serve as ground truth, and the whole corpus must be free of real personal data and documented well enough to survive the regulatory scrutiny of Sections 2.1 and 2.2. A methodology that produces construction-true labels at generation time, validates them with a learned detector, and ships the evidence alongside the data has done work the buyer would otherwise have to perform or leave undone and absorb as risk. The token-generation economics determine whether a defensible dataset can be developed and produced affordably — and the measured \$0.58 and \$3.60 ten-thousand-record runs show that it can, on either substrate. The attestation layer determines whether the resulting dataset is worth more to a regulated buyer than the raw tokens beneath it. The two substrates make the first true on either a local or a cloud footing; the production layer, identical across both, makes the second true regardless of which substrate generated the data.

10 Limitations

A methodology that asks regulated buyers to trust its output earns that trust partly by stating plainly what it does not and cannot guarantee. This section names the limitations of the work honestly, each followed where applicable by its mitigation or the path to addressing it, without overstating either the problem or the remedy. The limitations fall into three groups: the bounds of the de-identification measurements, the bounds of the pattern-based components, and the bounds of the substrates and the evidence.

10.1 Injected Value Fidelity

The clearest and most actionable limitation is in the fidelity of the injected span values. The detector evaluation of the ten-thousand-record medical run reported a value-validity tally of 42,055 well-formed values against 7,289 that were not — approximately 14.8% of the 49,344 injected span values failed format validation for their declared type. Because the de-identification record’s labels are correct by construction, this is not a labeling error: the spans are located and typed correctly, but a meaningful minority of the injected values are not well-formed exemplars of their type — a date that does not parse cleanly, an identifier that does not match its expected format, and similar shape failures. For a de-identification training corpus this is a real quality limitation, because a model learning from malformed exemplars of a type learns a distorted picture of what that type looks like.

The value generators in `providers.py` are the locus of both the problem and the fix: because the values are generated in code at known positions, tightening the generators to emit strictly well-formed values for every type is a bounded engineering task rather than an open research problem, and the value-validity metric already in the evaluation report gives a direct measurement to optimize against. This is named here as the single clearest target for the next round

of improvement, and it is reported rather than omitted precisely because the methodology’s posture is to disclose measured residual defects rather than suppress them.

10.2 The Macro-Averaged Detector Score

The detector evaluation carries a micro-averaged F1 of 0.964 and a macro-averaged F1 of 0.588, and the gap between them is a limitation of interpretation that must be stated rather than buried. As Section 7 explains, the micro-average weights each of the 49,344 spans equally and reflects detector performance on the data that actually exists, where every meaningful-support type clears an F1 of 0.95. The macro-average weights each type equally regardless of support, so the several near-empty types in the report — a single-span type, a three-span type, and two types with no ground-truth spans present in this medical corpus — each contribute an F1 of 0.000 and pull the macro figure down to 0.588.

The limitation is therefore twofold. First, the macro-averaged figure cannot be read as a measure of detector quality on this dataset; it is an averaging artifact of types that are effectively absent. Second, and more substantively, the presence of those near-zero-support types indicates that this single medical run does not exercise the full identifier vocabulary — some types that the methodology supports simply did not arise in a medical de-identification corpus, so the detector’s performance on them is untested by this run rather than poor. A complete characterization of the detector across all supported types would require runs in domains that generate those types at meaningful frequency, which the present evidence does not include.

10.3 Pattern-Based Detection and Masking Cannot Be Proven Complete

The masking layer, the template-leak gate, and the regex half of the hybrid detector are pattern-based, operating on free-form model output with compiled regular expressions and keyword sets. This imposes a hard epistemic limit: pattern matching cannot be proven complete. The template-leak episode of Section 6 is the direct evidence — a gate that recognized only one skeleton style silently accepted the others until it was widened, and the honest reading of that history is that a further, as-yet-unseen skeleton style, or an unanticipated personal-data shape, could exist in output the pipeline has not yet generated. The widened template-leak gate and the always-mask floor make such an escape less likely, and the hybrid detector’s learned tier covers the free-text entities that no regular expression can match, but none of these makes the guarantee absolute. The honest claim is bounded accordingly: the masking and template-leak gates are validated against the failure modes encountered to date and are strengthened when a new one is found, not proven exhaustive against all possible output.

10.4 Threshold Calibration Is Empirical, Not Adaptive

The hallucination threshold — the unified 0.45 cosine floor documented in Section 4 — was calibrated empirically against the embedding model’s behavior on this pipeline’s output distribution, and it does not self-adjust. The recalibration from the earlier 0.55/0.58 regime to 0.45 was itself driven by a shift in the output profile toward completed records, and a further shift — a new domain with a different vocabulary profile, a change of embedding model, or a change of generation model — could render the current floor miscalibrated. Recalibration is not automated. Its mitigation is that it is cheap and configuration-driven: a small calibration run surfaces the empirical similarity distribution for the new configuration, from which a corrected floor is read and set through a single configuration value with no code change, exactly as the current floor is set. The limitation is that this is a manual step that must be repeated when the embedding model, generation model, or domain profile changes materially.

10.5 Substrate and Scale Bounds

The local substrate is bounded by a single GPU. Veritrukt Studio runs on one RTX 4090, and its throughput is fixed by that hardware — approximately 561 accepted records per hour on the ten-thousand-record medical run — with no horizontal scaling available locally. A larger local deliverable does not run faster; it runs longer, as the roughly 17.84-hour Studio run demonstrates. This is the structural limit that motivates the Cloud substrate, and within the local substrate it is not removable. The mitigation is the Cloud substrate itself, whose measured 3.38-hour run on the same deliverable is documented in Section 9; the limitation is simply that local generation cannot scale beyond what one GPU delivers.

The evidence in this paper is also bounded in scale and scope. The two production runs that anchor the results are ten-thousand-record runs in the medical domain, on the de-identification record type. The methodology supports seven domains and two record types, but the quantitative results presented here characterize one domain and one record type at the ten-thousand-record scale. Behavior in the other domains, on the instruction fine-tuning record type at production scale, and at substantially larger record counts, is consistent with the architecture but is not demonstrated by the specific runs documented here. Stating this bound is itself part of the methodology’s posture: the results establish what the two measured runs establish — a validated, reconciled, construction-true medical de-identification dataset on each substrate — and no more than that. The same transparency that reports a residual value-validity shortfall rather than claiming an unqualified success applies to the scope of the evidence, so that what is measured and what is merely supported by the architecture are never conflated.

11 Appendix

This appendix provides reference material for readers verifying or reproducing the work: the operative configuration values, an inventory of the runs cited in the paper, the environment specification for both substrates, and a glossary of technical terms. All values are drawn from the resolved configuration and source of the runs documented in Section 8.

11.1 Configuration Reference

The methodology is configuration-driven; the values below are the operative settings for the production runs, read from the resolved run configuration.

Table 11.1. *Hallucination gate. The 0.45 floor is applied uniformly across the default and every per-domain threshold, on both substrates.*

Parameter	Value
Default threshold	0.45
Per-domain thresholds	0.45 (all seven domains)
Borderline margin	± 0.05 cosine of active threshold
Embedding batch size	22

Table 11.2. *Generation parameters.*

Parameter	Value
Temperature	0.3
Top-p	0.92
Repetition penalty	1.1

Parameter	Value
Max new tokens	1024
Over-generation ratio	0.25
Studio batch size	22
Cloud batch size	128

Table 11.3. *Masking, de-identification, and placeholder control.*

Parameter	Value
Active masking profile	gdpr
De-identification detector	hybrid (regex + GLiNER2)
Labeling mode	construction-true
Target placeholders per record	2.5
Minimum placeholders per record	1
Maximum placeholders per record	6

11.2 Run Inventory

Table 11.4. *Runs referenced in this paper. The two ten-thousand-record medical de-identification runs anchor the results in Sections 8 and 9; the calibration and recovery-arc runs support the analyses in Sections 4 and 6.*

Run ID / label	Substrate / GPU	Records	Role in paper
run_studio_20260629_212754	Studio / RTX 4090	10,016	Production de-id run; §4 gate validation, §6 recovery endpoint, §7 detector evaluation, §8/§9 results
run_cloud_20260630_160703	Cloud / A10G	10,064	Production de-id run; §8 substrate comparison, §9 cost/throughput comparison
run_studio_20260619_020714	Studio / RTX 4090	~1,000	Detector-tier calibration (§5 — removal of AC-COUNT_NO/SWIFT_BIC from the learned tier)

Run ID / label	Substrate / GPU	Records	Role in paper
Recovery-arc runs (Jun 2026)	Studio & Cloud	100-125 each	\$6 yield-recovery arc (document-type, threshold, value-packet, repair fixes)

11.3 Environment Specification — Local Substrate (Veritrukt Studio)

Table 11.5. *Local generation, embedding, and evaluation environment. The generation and embedding models are quantized in-house. Detection and masking use a hand-written compiled-regex pattern registry; the hybrid detector adds the learned GLiNER2 tier.*

Component	Specification
GPU	NVIDIA RTX 4090 (24 GB)
Host environment	WSL2
PyTorch	2.4.1 + cu121
Transformers	4.57.6
Generation model	Mistral-Nemo-Instruct-2407, 4-bit BitsandBytes (in-house)
Embedding model	Snowflake Arctic Embed L v2.0, 4-bit (in-house)
PII detection & masking	Hand-written compiled-regex pattern registry (Python re)
Learned NER tier	GLiNER2 (fastino/gliner2-large-v1), confidence threshold 0.5,
Backend	include_spans=True LocalHFBackend (in-process generation)

11.4 Environment Specification — Cloud Substrate (Veritrukt Cloud)

Table 11.6. *Remote Modal vLLM serving environment. Generation runs remotely; the quality gate, masking, de-identification validation, and evaluation remain on the local substrate (Table 11.5).*

Component	Specification
Modal app name	syntheval-cloud-vllm
Serving class	MistralNemoAWQVllm
GPU	NVIDIA A10G (24 GB)
Serving engine	vLLM
Model	Mistral-Nemo-Instruct-2407 AWQ (in-house)
max_model_len	2560
Generation batch size	128
Workers	1 (sequential batch dispatch)
Per-call RPC timeout	600 s
Backend	ModalVLLMBackend (remote generation)

Component	Specification
Evaluation / embedding	Local RTX 4090 (as Table 11.5)

11.5 Glossary

Table 11.7. *Technical terms used in this paper.*

Term	Definition
AWQ	Activation-aware Weight Quantization; the 4-bit quantization of the cloud generation model.
BitsandBytes NF4	The 4-bit NormalFloat quantization of the local generation and embedding models.
Construction-true labeling	De-identification labels that are correct by construction because the personal-data values are injected at known character offsets, rather than applied by a fallible post-hoc tagging pass.
Cosine similarity	The measure of instruction-output semantic correspondence used by the hallucination gate.
GLiNER2	The learned named-entity-recognition model (fastino/gliner2-large-v1) that forms the learned tier of the hybrid detector, recovering free-text entities regex cannot match.
Hybrid detector	The de-identification detector combining the regex detector (structured types, precedence on overlaps) with the GLiNER2 learned tier (free-text entities).
Macro-average	A per-type F1 average weighting every type equally regardless of support; sensitive to low-support types.
Micro-average	An F1 computed by pooling true/false positives and negatives across all spans; weights each span equally.
Skeleton / template leak	Fill-in-the-blank stub output ([Your Name], <INSERT_X>, [Sharp/Dull/Aching]) emitted in place of completed content; rejected by the template-leak gate.
Span	A labeled personal-data element with a character offset, length, and type — the unit of a de-identification record’s ground truth.
Value packet	Pre-generated concrete non-PII narrative values handed to the model so it completes records rather than skeletonizing them.
Yield	The proportion of generated records accepted after the full quality and compliance cascade.

12 References

Adler, B., Agarwal, N., Aithal, A., Anh, D. H., Bhattacharya, P., Brundyn, A., Casper, J., Catanzaro, B., Clay, S., Cohen, J., Das, S., Dattagupta, A., Delalleau, O., Derczynski, L., Dong, Y., Egert, D., Evans, E., Ficek, A., Fridman, D., . . . You, J. (2024). *Nemotron-4 340B technical report* (arXiv:2406.11704). arXiv. <https://arxiv.org/abs/2406.11704>

AI Lawsuit Tracker. (2026, April). *AI lawsuit tracker (2026): OpenAI, NYT & 164 other AI copyright litigation cases*. <https://ailawsuittracker.com/>

Anthropic. (2026, January 22). *Claude's new constitution*. <https://www.anthropic.com/news/claude-new-constitution>

Anthropic. (2026, May). *Pricing — Claude API docs*. <https://platform.claude.com/docs/en/about-claude/pricing>

Authors Alliance. (2026, May 14). *Bartz v. Anthropic fairness hearing: Final reminder, 91.3% claims rate, and updates from the docket*. <https://www.authorsalliance.org/2026/05/14/bartz-v-anthropic-fairness-hearing-final-reminder-91-3-claims-rate-and-updates-from-the-docket/>

Authors Guild. (2025, September 26). *Bartz v. Anthropic settlement: What authors need to know*. <https://authorsguild.org/advocacy/artificial-intelligence/what-authors-need-to-know-about-the-anthropic-settlement/>

Berberich, M. (2025, June 13). *AI and data protection: Using AI systems in compliance with the GDPR*. SRD Rechtsanwälte. <https://www.srd-rechtsanwaelte.de/en/blog/privacy-ai-gdpr>

DeepSeek-AI. (2025). *DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning* [Model release and technical documentation]. GitHub. <https://github.com/deepseek-ai/DeepSeek-R1>

European Parliament & Council of the European Union. (2016). *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)*. Official Journal of the European Union, L 119, 1–88. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>

Gemini Team, Google. (2025, June 16). *Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities*. Google DeepMind. https://storage.googleapis.com/deepmind-media/gemini/gemini_v2_5_report.pdf

Guan, M. Y., Joglekar, M., Wallace, E., Jain, S., Barak, B., Helyar, A., Dias, R., Vallone, A., Ren, H., Wei, J., Chung, H. W., Toyer, S., Heidecke, J., Beutel, A., & Glaese, A. (2024, December 20). *Deliberative alignment: Reasoning enables safer language models*. OpenAI. <https://openai.com/index/deliberative-alignment/>

IAPP. (2025, November 5). *How to train AI lawfully?* International Association of Privacy Professionals. <https://iapp.org/news/a/how-to-train-ai-lawfully->

Latham & Watkins. (2025, November 13). *Getty Images v. Stability AI: English High Court rejects secondary copyright claim*. <https://www.lw.com/en/insights/getty-images-v-stability-ai-english-high-court-rejects-secondary-copyright-claim>

Mayer Brown. (2025, November 20). *Getty Images v Stability AI: What the High Court's decision means for rights-holders and AI developers*. <https://www.mayerbrown.com/en/insights/publications/2025/11/getty-images-v-stability-ai-what-the-high-courts-decision-means-for-rights-holders-and-ai-developers>

Meta AI. (2024, July 23). *Introducing Llama 3.1: Our most capable models to date*. <https://ai.meta.com/blog/meta-llama-3-1/>

Mistral AI. (2025). *Mistral-Nemo-Instruct-2407* [Model card]. Hugging Face. <https://huggingface.co/mistralai/Mistral-Nemo-Instruct-2407>

Modal Labs. (2026). *Modal: Serverless GPU compute platform* [Computer software documentation]. <https://modal.com/docs>

National Law Review. (2026, January). *OpenAI loses privacy gambit: 20 million ChatGPT logs likely headed to copyright plaintiffs*. <https://natlawreview.com/article/openai-loses-privacy-gambit-20-million-chatgpt-logs-likely-headed-copyright>

Norton Rose Fulbright. (2025). *Recent regulatory developments in training artificial intelligence (AI) models under the GDPR*. <https://www.nortonrosefulbright.com/en/knowledge/publications/a80c5e1d/recent-regulatory-developments-in-training-artificial-intelligence-ai-models-under-the-gdpr>

Norton Rose Fulbright. (2026, March 13). *AI in litigation series: An update on AI copyright cases in 2026*. <https://www.nortonrosefulbright.com/en/knowledge/publications/ce8eaa5f/ai-in-litigation-series-an-update-on-ai-copyright-cases-in-2026>

Oklahoma Gas & Electric. (2026). *Residential electric service billing statement* [Utility bill, confidential primary source on file with the author].

OpenAI. (2026, May). *OpenAI API pricing*. <https://openai.com/api/pricing/>

Orrick, Herrington & Sutcliffe. (2025, March). *The European Data Protection Board shares opinion on how to use AI in compliance with GDPR*. <https://www.orrick.com/en/Insights/2025/03/The-European-Data-Protection-Board-Shares-Opinion-on-How-to-Use-AI-in-Compliance-with-GDPR>

PBH Applied Systems LLC. (2026a). *Veritruct Studio production run telemetry: Reference run_studio_20260629_212754* [Run manifest, evaluation report, and GPU power telemetry]. Unpublished raw data.

PBH Applied Systems LLC. (2026b). *Veritruct Cloud production run telemetry: Reference run_cloud_20260630_160703* [Run manifest, evaluation report, and Modal cost record]. Unpublished raw data.

PPC Land. (2026, March 29). *GDPR's AI training legal battle: Regulators converge but still clash*. <https://ppc.land/gdprs-ai-training-legal-battle-regulators-converge-but-still-clash/>

Ropes & Gray. (2026, January 14). *Getty Image loses copyright infringement claim against Stability AI in UK's first-of-its-kind ruling*. <https://www.ropesgray.com/en/insights/viewpoints/102lvxe/getty-image-loses-copyright-infringement-claim-against-stability-ai-in-uks-first>

Skadden, Arps, Slate, Meagher & Flom. (2025, June 25). *CNIL clarifies GDPR basis for AI training — but it's just one part of the compliance picture*. <https://www.skadden.com/insights/publications/2025/06/cnil-clarifies-gdpr-basis-for-ai-training>

Snowflake. (2024, December 4). *Snowflake's Arctic Embed 2.0 goes multilingual: Empowering global-scale retrieval with inference efficiency and high-quality retrieval*. Snowflake Engineering Blog. <https://www.snowflake.com/en/engineering-blog/snowflake-arctic-embed-2-multilingual/>

U.S. Department of Health and Human Services. (2012). *Guidance regarding methods for de-identification of protected health information in accordance with the Health Insurance Portability and Accountability Act (HIPAA) Privacy Rule*. Office for Civil Rights. <https://www.hhs.gov/hipaa/for-professionals/privacy/special-topics/de-identification/index.html>

vLLM Project. (2026). *vLLM: Easy, fast, and cheap LLM serving for everyone* [Computer software documentation]. <https://docs.vllm.ai>

Wiggers, K. (2024, December 24). *The promise and perils of synthetic data*. TechCrunch. <https://techcrunch.com/2024/12/24/the-promise-and-perils-of-synthetic-data/>

Wilson Sonsini Goodrich & Rosati. (2026, March 31). *OpenAI prevails in landmark Italian AI and GDPR enforcement case*. <https://www.wsgr.com/en/insights/openai-prevails-in-landmark-italian-ai-and-gdpr-enforcement-case.html>

Wolters Kluwer. (2025, November 10). *The Bartz v. Anthropic settlement: Understanding America's largest copyright settlement*. Kluwer Copyright Blog. <https://legalblogs.wolterskluwer.com/2025/11/10/the-bartz-v-anthropic-settlement-understanding-americas-largest-copyright-settlement/>

uwer.com/copyright-blog/the-bartz-v-anthropic-settlement-understanding-americas-largest-copyright-settlement/

Yu, P., Merrick, L., Nuti, G., & Campos, D. (2024). *Arctic-Embed 2.0: Multilingual retrieval without compromise* (arXiv:2412.04506). arXiv. <https://arxiv.org/abs/2412.04506>

Zaratiana, U., Tomeh, N., Holat, P., & Charnois, T. (2024). *GLiNER: Generalist model for named entity recognition using bidirectional transformer* (arXiv:2311.08526). arXiv. <https://arxiv.org/abs/2311.08526>